Extending Multibody Simulation with CADENCE: Closed-Form Dynamics from URDF Models for Model-Based Control and Trajectory Optimization of Serial Manipulators

Ahmed Askar[1,2]

[1]Department of Mechatronics and Robotics Engineering, School of Innovative Design Engineering, Egypt-Japan University of Science and Technology (E-JUST), New Borg El-Arab City, Alexandria, Egypt
[2]Department of Production Engineering and Mechanical Design, Faculty of Engineering, Mansoura University, Mansoura, Egypt

October 30, 2026

Abstract

Multibody simulators based on the Unified Robot Description Format (URDF) predict robot motion accurately but keep their equations of motion implicit, which limits model-based control and trajectory optimization. We present CADENCE (Closed-form Analytical Dynamics Extraction for Nonlinear Control Extension), an automated pipeline that extends such simulators with closed-form, differentiable dynamics generated directly from URDF models of serial manipulators. The pipeline synthesizes Denavit–Hartenberg parameters, resolving frame ambiguities, projects corrupted CAD inertia tensors onto the physically realizable set, and computes Coriolis and centrifugal terms as a directional derivative of the generalized momentum, with cost bounded by a constant multiple of the mass-matrix cost. Forward dynamics uses symbolic Cholesky factorization instead of matrix inversion. Across chains of 3 to 14 degrees of freedom, graph size scales as $\mathcal{O}(n^{2.29})$ rather than $\mathcal{O}(n^{3.25})$ for Christoffel expansions, cutting forward-dynamics instructions by 55.4% for a 6-DOF arm and 75.5% at 14 DOF. Models of four industrial manipulators agree with MuJoCo and Simscape, and compiled C forward dynamics runs in 78.7 μs. The exact derivatives enable constrained trajectory optimization, converging on all 15 test problems per robot (100% convergence in 7.9–15.6 iterations, reducing iterations by 95.6%–96.8% and accelerating solve time up to 14.39× over Quasi-Newton L-BFGS), and computed-torque control that maintains robust tracking under moderate parameter mismatch, while large unmodeled payloads motivate adaptive parameter calibration.

Introduction

Closed-form kinematic and dynamic models remain the cornerstone of robot analysis, nonlinear control, and robotics education. Minimal parameterizations, most notably the standard Denavit–Hartenberg (DH) convention [1–4], combined with Euler–Lagrange mechanics, establish an explicit relationship between kinematic geometry, inertial properties, and system behavior. This analytical transparency provides the structural insight required for dynamic analysis, formal stability assessment, and systematic model-based controller synthesis. In contemporary robotics practice, however, development is increasingly driven by software ecosystems such as ROS [5], MoveIt [6], Gazebo [7], PyBullet [8], MuJoCo [9], and Simscape Multibody [10]. These platforms predominantly describe articulated mechanisms using the Unified Robot Description Format (URDF) [11, 12], which represents robots as trees of rigid bodies connected through local coordinate transformations. Although highly effective for software integration, motion planning, collision detection, and numerical simulation, this representation differs fundamentally from the minimal analytical parameterizations employed in classical robot modeling. As a result, a persistent gap has emerged between simulation-oriented robot descriptions and the explicit analytical models required for rigorous dynamics analysis and control design [13, 14].

This gap is further widened by the computational paradigms underlying modern simulation engines. Rather than deriving explicit symbolic equations of motion, contemporary frameworks typically employ constrained optimization techniques [9] or recursive rigid-body algorithms, most notably Featherstone's Articulated Body Algorithm (ABA) and Recursive Newton–Euler Algorithm (RNEA) [15, 16]. By exploiting the tree structure of articulated mechanisms, these methods enable highly efficient forward and inverse dynamics computations while

supporting robust contact and collision handling. Their efficiency, however, comes from avoiding the construction of global analytical models. Consequently, users interact with the simulator primarily through numerical state-transition operators that map joint positions, velocities, and actuation inputs to accelerations, whereas the underlying dynamic relationships remain embedded within the computational framework and are not exposed in an explicit analytical form. This abstraction is advantageous for simulation and real-time execution but limits direct access to the mathematical structure required for theoretical analysis, stability proofs, and model-based controller design.

The absence of explicit dynamic equations poses a significant challenge for model-based control and trajectory optimization, both of which require the equations of motion and their derivatives. High-fidelity environments such as MSC ADAMS, Simscape Multibody, Gazebo, and MuJoCo accurately perform dynamic simulation, but they generally do not expose the nonlinear equations required for systematic controller synthesis [17, 18]. Where control-design utilities are available, they typically rely on local Jacobian linearizations around fixed operating points, which limits their validity to a narrow region of operation [17, 18]. Performance therefore deteriorates during highly dynamic maneuvers, in which configuration-dependent inertia variations and velocity-dependent Coriolis and centrifugal effects dominate. Without globally valid analytical models, practitioners often resort to model-free architectures, most commonly decentralized PID controllers, whose inability to compensate for nonlinear dynamic couplings leads to reduced tracking accuracy, increased phase lag, and actuator chattering under demanding conditions [19, 20].

Data-driven model discovery, particularly Sparse Identification of Nonlinear Dynamics (SINDy), has been explored as a way to recover governing equations directly from simulated trajectories [17, 18, 21, 22]. Although successful in low-dimensional systems, these approaches are difficult to extend to articulated manipulators. The trigonometric complexity of multi-degree-of-freedom kinematics produces large candidate libraries and poorly conditioned regression problems, and physically meaningful models typically require extensive excitation trajectories, library reduction strategies, and physics-constrained optimization [17, 18]. These difficulties motivate methods that derive exact and physically consistent equations of motion directly from CAD- and URDF-based robot descriptions, instead of reconstructing them indirectly from simulated data.

The demand for explicit analytical models is further amplified by the emergence of industrial digital twins [13, 14, 23]. A URDF model captures only the nominal geometry and inertial properties of a robot, and it cannot represent the manufacturing tolerances, assembly deviations, payload variations, or structural compliance encountered in deployed hardware [19, 24]. Bridging this simulation-to-reality gap requires calibrating kinematic and dynamic parameters from experimental measurements [21, 23]. Such calibration is most naturally formulated within gradient-based optimization frameworks, which in turn require differentiable analytical models that can be embedded in tools such as CasADi [25].

Existing solutions address only part of this need. Available URDF conversion tools either rely on auxiliary dummy frames, fail on general branched trees, or stop at kinematics without propagating inertial properties into the synthesized DH frames [26–28]. A general framework that transforms industrial CAD and URDF descriptions into explicit and physically consistent analytical dynamic models therefore remains largely unavailable [11, 28]. Three technical obstacles explain this gap. First, a URDF joint is an unconstrained six-degree-of-freedom transformation referenced to arbitrary CAD origins, whereas the DH convention imposes two strict geometric rules on the frame assignment and leaves it non-unique whenever consecutive joint axes are parallel or collinear [2, 28, 29]. Second, link masses, centers of mass, and inertia tensors are defined relative to CAD link origins and must be transformed exactly into the DH frames; moreover, CAD-derived and historical inertia tensors frequently violate rigid-body realizability, that is, positive definiteness and the triangle inequality on the principal moments [11, 18, 20], which causes numerical breakdowns in dynamic simulation. Third, deriving closed-form Euler–Lagrange equations with computer algebra suffers from severe expression swell: forming the n^3 Christoffel symbols and symbolically inverting the mass matrix generate expressions that grow rapidly beyond six degrees of freedom and preclude real-time execution. Recursive spatial-vector libraries avoid this swell through procedural C/C++ routines, but they do not expose compressed, DH-consistent symbolic graphs across the MATLAB, Simulink, and Python ecosystems [16].

This paper addresses these obstacles with CADENCE (Closed-form Analytical Dynamics Extraction for Nonlinear Control Extension), a deterministic, fully automated framework that converts raw, uncalibrated industrial URDF models into physically consistent, closed-form analytical dynamic models (Figure 1). The aim is to extend multibody simulation, not to replace it: the generated models are verified against established physics engines and operate alongside them, so that a single URDF description serves both as the simulated plant and as the source of the equations needed for model-based control, trajectory optimization, and digital-twin calibration. The framework synthesizes DH-consistent kinematic descriptions, propagates and validates the inertial properties, derives the closed-form Euler–Lagrange equations of motion, and reformulates the dynamic computations through algorithmic differentiation to overcome the scalability limits traditionally associated with symbolic robot dynamics. We test the hypothesis that the combinatorial growth which has restricted closed-form Lagrangian mechanics to low-dimensional manipulators is not intrinsic to analytical mechanics, but an artifact of the Christoffel-tensor formulation, and that removing it reduces the cost of the sensitivity evaluations that dominate gradient-based

trajectory optimization, including its receding-horizon form, nonlinear model predictive control (NMPC), and calibration. We focus on serial manipulators, the dominant class of industrial robots and the backbone of manufacturing, assembly, and material-handling tasks; the underlying principles of automated model extraction, inertial parameter propagation, and differentiable symbolic dynamics provide a foundation for future extension to mobile manipulators, legged systems, humanoid robots, and hybrid kinematic structures.

The main contributions of this paper are as follows:

- **Automated Kinematic and Inertial Synthesis:** An automated framework that extracts DH parameters from a URDF file while resolving the arbitrary frame assignments inherent in URDF descriptions.
- **Momentum-Based Computation of Coriolis and Centrifugal Terms:** A generalized-momentum-based formulation that eliminates the expression growth of the Christoffel-symbol approach while preserving mathematical equivalence, passivity, and computational complexity comparable to mass matrix evaluation.
- **Inverse-Free Forward Dynamics:** A symbolic factorization-based solution that exploits the positive definiteness of the mass matrix to avoid explicit inversion and produce compact, differentiable expressions for numerical optimization.
- **Extending Simulation with Model-Based Control and Optimization:** A demonstration that the generated models provide exact derivatives for trajectory optimization and model-based control, including predictive control under actuator and obstacle constraints.
- **Multi-Platform Deployment and Verification:** A unified model representation deployed across symbolic MATLAB and compiled C implementations and automatically verified against MuJoCo and Simscape on four industrial manipulators.

The CADENCE Pipeline Architecture and Workflow

To bridge the gap between simulation-oriented robot descriptions and model-based control synthesis, we develop **CADENCE** (**C**losed-form **A**nalytical **D**ynamics **E**xtraction for **N**onlinear **C**ontrol **E**xtension), an automated, end-to-end computational pipeline that ingests uncalibrated industrial robot descriptions and emits exact, differentiable dynamic models. As illustrated in Figure 1, CADENCE is structured into three consecutive computational stages:

1. **Inputs to CADENCE:** The pipeline takes as input a standard URDF XML file, which defines the robot as an unconstrained tree of links and joints with arbitrary CAD-derived reference origins, unaligned coordinate orientations, manufacturer-specific actuation signs, and raw (potentially unphysical) inertia tensors. For control and trajectory optimization, task-level specifications are additionally provided, including joint position and velocity limits $[\boldsymbol{x}_{\min}, \boldsymbol{x}_{\max}]$, actuation torque bounds $[\boldsymbol{u}_{\min}, \boldsymbol{u}_{\max}]$, end-effector reference trajectories $\boldsymbol{x}_{\text{ref}}(t)$, and geometric obstacle coordinates $(\boldsymbol{p}_{\text{obs}}, r_{\text{obs}})$.
2. **Core Model Extraction and Dynamics Engine:** CADENCE parses the topological tree into an active serial chain, contractively absorbing intermediate fixed links. It then synthesizes standard DH parameters through an exhaustive four-case geometric classification (Case 1: general skew, Case 2: intersecting, Case 3: parallel, and Case 4: collinear axes) while automatically identifying the joint-axis actuation parity vector $\boldsymbol{S} \in \{-1, +1\}^n$. Link inertia tensors are mapped to the synthesized DH frames and projected onto the physically realizable set, enforcing positive definiteness and the rigid-body triangle inequality. The sanitized kinematic and inertial parameters are then assembled into the closed-form mass metric $\boldsymbol{M}(\boldsymbol{q})$ and gravitational vector $\boldsymbol{G}(\boldsymbol{q})$. Velocity-dependent Coriolis and centrifugal effects are derived as an exact directional derivative of the generalized momentum, $\boldsymbol{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \left(\frac{\partial(\boldsymbol{M}\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}} - \frac{1}{2}\nabla_{\boldsymbol{q}}(\dot{\boldsymbol{q}}^T \boldsymbol{M}\dot{\boldsymbol{q}})$, bypassing the combinatorial explosion of n^3 Christoffel symbols. Finally, forward dynamics are resolved via symbolic Cholesky factorization $\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{R}(\boldsymbol{q})^T \boldsymbol{R}(\boldsymbol{q})$ without explicit matrix inversion.
3. **Outputs and Multi-Environment Deployment:** CADENCE outputs: (i) differentiable symbolic CasADi computation graphs and self-contained, heap-free ANSI C99 source code for forward and inverse dynamics along with exact state-transition Jacobians $\boldsymbol{A}_k, \boldsymbol{B}_k$ and second-order Lagrangian Hessians $\nabla^2_{\boldsymbol{ww}}\mathcal{L}$; (ii) native executable models deployed across three execution environments—`MATLAB-Sym` for transparent algebraic inspection and Lyapunov proofs, `MATLAB-CasADi` for Simscape Multibody co-simulation and Model-Based Design, and `Python-MuJoCo` for high-rate servo rollouts ($>$ 10 kHz), Computed Torque Control (CTC), offline trajectory optimization, and receding-horizon NMPC; and (iii) automated certificates generated by an integrated five-stage verification procedure confirming kinematic, dynamic, and simulator parity.

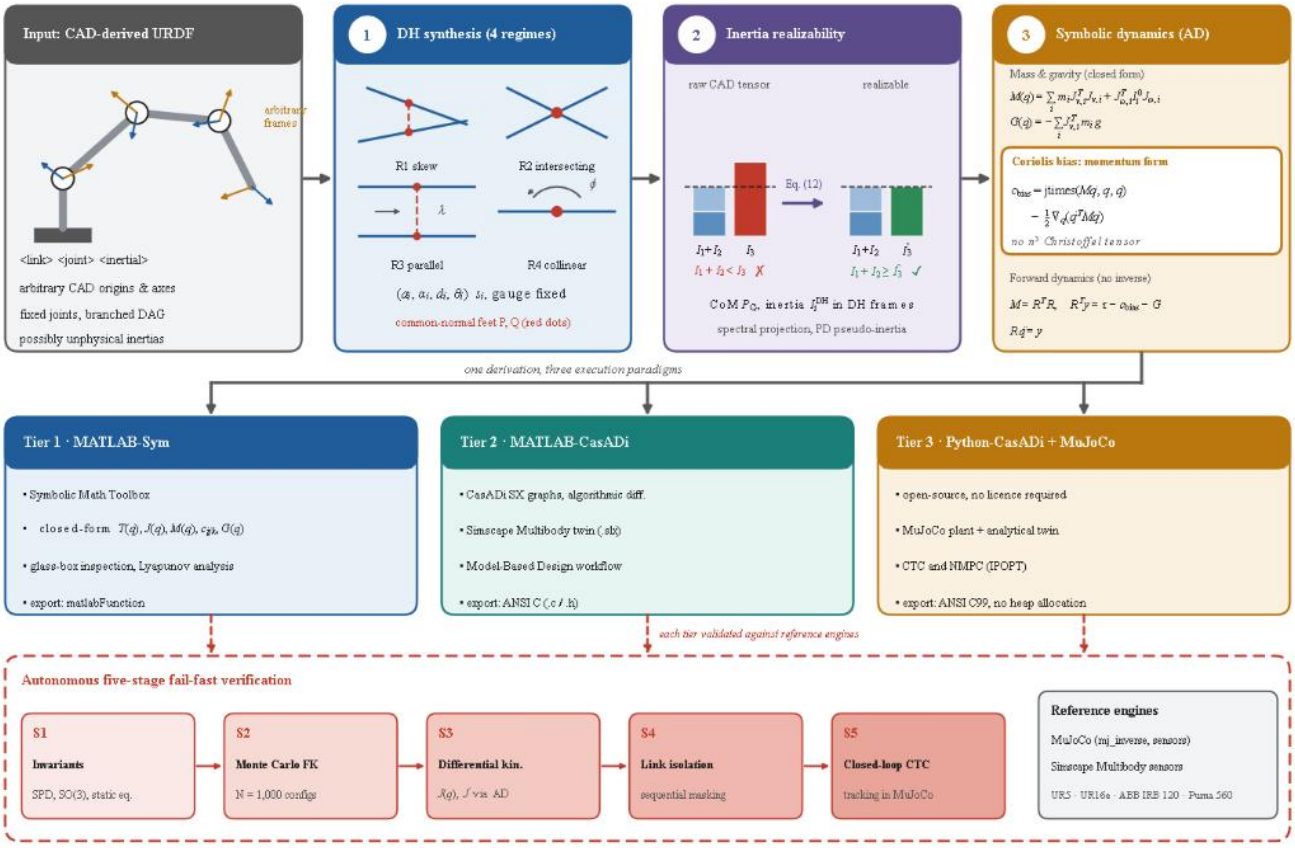


Figure 1: **Overview of the CADENCE framework.** Automated transformation of raw URDF descriptions into closed-form, differentiable dynamics models for model-based control, trajectory optimization, and simulation integration across `MATLAB-Sym`, `MATLAB-CasADi`, and `Python-MuJoCo`. Detailed inputs, core stages, and outputs are presented in Section 1.

The remainder of the paper is organized as follows. Section 1 presents the analytical methodology of CADENCE, detailing DH synthesis across the four geometric cases, inertia sanitization, momentum-based Coriolis evaluation, and symbolic Cholesky forward dynamics. Section 2 establishes the model-based control framework, trajectory optimization, and multi-environment software deployment. Section 3 formalizes the multi-stage verification suite across four benchmark manipulators. Section 4 reports empirical results on derivative fidelity, multi-DOF scalability, offline trajectory optimization, closed-loop tracking under parametric mismatch, and real-time NMPC. Finally, Section 5 discusses the implications of CADENCE for simulation, optimal control, and robotics education, followed by concluding remarks.

1 Analytical Framework for Kinematic and Dynamic Modeling of Serial Manipulators

This section presents an automated framework for generating a complete analytical dynamic model from a URDF description. The framework automatically extracts a DH representation, verifies and corrects inertial properties, and derives the equations of motion in closed form. The resulting model is suitable for simulation, control, and trajectory optimization.

1.1 Denavit–Hartenberg Convention

The DH convention describes the pose of each link frame relative to its predecessor using four parameters [2, 3]. As illustrated in Fig. 2, consecutive frames are assigned such that

$$\text{Rule R1:} \quad \hat{\boldsymbol{x}}_i \perp \hat{\boldsymbol{z}}_{i-1}, \tag{1}$$

$$\text{Rule R2:} \quad \hat{\boldsymbol{x}}_i \text{ lies along the common normal intersecting } \hat{\boldsymbol{z}}_{i-1}, \tag{2}$$

so that $\hat{\boldsymbol{x}}_i$ coincides with the common normal between the joint axes $\hat{\boldsymbol{z}}_{i-1}$ and $\hat{\boldsymbol{z}}_i$. The relative pose is then fully characterized by the link length a_i, the link twist α_i, the link offset d_i, and the joint angle θ_i. These parameters

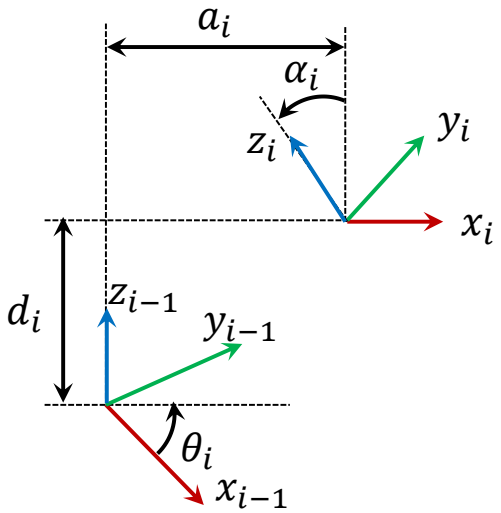


Figure 2: **Coordinate frames satisfying the DH constraints R1 and R2.** The link length a_i and twist α_i are measured along and about $\hat{\boldsymbol{x}}_i$, while the link offset d_i and joint angle θ_i are measured along and about $\hat{\boldsymbol{z}}_{i-1}$. Dashed lines indicate translated copies of the axes used to visualize distances and angles.

define the homogeneous transformation

$$\boldsymbol{A}_i = \mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i) = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{3}$$

where the joint variable enters through θ_i for a revolute joint and through d_i for a prismatic joint, while the remaining parameters remain constant.

DH parameterization is inherently non-unique because the DH geometric constraints do not uniquely determine all frame placements. For parallel joint axes, the common normal may be translated arbitrarily along the axes, introducing a translational degree of freedom. For collinear axes, any line perpendicular to the shared axis is a valid common normal, introducing an additional rotational degree of freedom about the axis. When consecutive axes intersect, the common normal is defined only up to its sign; reversing it changes the sign of the twist angle α_i and shifts the joint angle offset θ_i by π. The direction of each joint axis, which defines the positive sense of joint motion, may also be chosen arbitrarily, changing the sign convention of the corresponding joint variable. For prismatic joints, only the axis direction is specified, while the line itself may be positioned arbitrarily. Furthermore, the first frame is constrained only through its z-axis, and the terminal frame has no successor joint to constrain its placement, leaving additional freedom in their origins and x-axis orientations. These choices generate distinct DH parameterizations that nevertheless yield identical forward kinematics. Consequently, DH tables reported for the same manipulator often differ across textbooks, research articles, and manufacturer documentation [2–4]. An additional source of variation is the choice between the standard and modified DH conventions, which affects the form of the link transformation but is independent of the freedoms described above. The present work adopts the standard DH convention and defines a reproducible procedure for selecting among equivalent frame assignments while preserving the manipulator kinematics.

1.2 Kinematic Chain Extraction and Denavit–Hartenberg Parameter Synthesis

Dynamic model generation from a URDF description begins by constructing a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of links and $\mathcal{E}$ the set of joints. Each joint $k \in \mathcal{E}$ is characterized by a parent-to-child transformation $\boldsymbol{T}_{\mathrm{joint},k} \in SE(3)$ and a local actuation axis $\hat{\boldsymbol{u}}_k \in S^2$ [28].

Assume that, at the home configuration ($\boldsymbol{q} = \boldsymbol{0}$), $\boldsymbol{O}_{i-1}, \boldsymbol{O}_i \in \mathbb{R}^3$ are the origins of consecutive active joints and $\hat{\boldsymbol{z}}_{i-1}, \hat{\boldsymbol{z}}_i \in S^2$ are their corresponding unit joint axes, all expressed in the base frame $\{\mathcal{F}_0\}$. Defining the origin offset $\Delta\boldsymbol{o}_i = \boldsymbol{O}_i - \boldsymbol{O}_{i-1}$ and the axis alignment metric $\delta_\times = \|\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i\|_2$, the geometric separation between consecutive joint axes is quantified by

$$d_{\min} = \frac{|\Delta\boldsymbol{o}_i \cdot (\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i)|}{\delta_\times} \quad (\delta_\times > \epsilon_\parallel), \qquad d_\perp = \|\Delta\boldsymbol{o}_i - (\Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_{i-1})\hat{\boldsymbol{z}}_{i-1}\|_2 \quad (\delta_\times \le \epsilon_\parallel), \tag{4}$$

where $d_{\min}$ denotes the shortest orthogonal distance between non-parallel axes and $d_\perp$ the perpendicular distance between parallel axes.

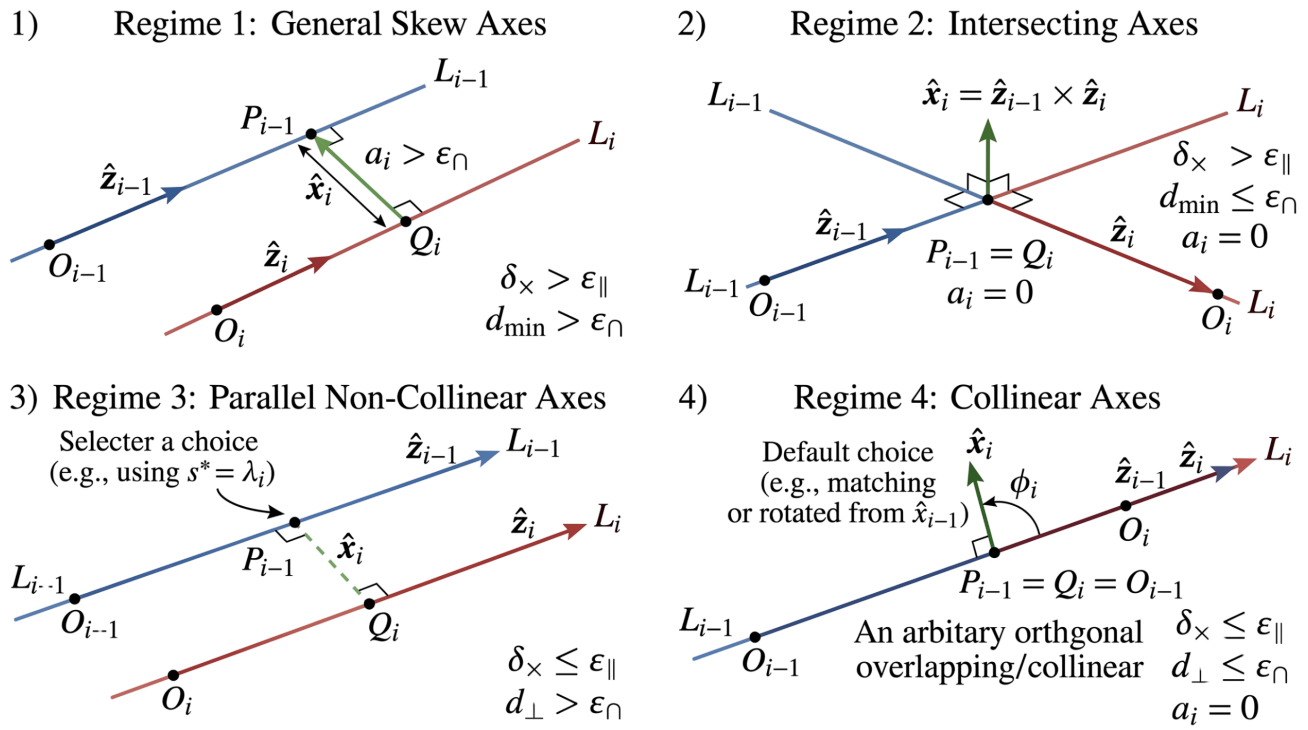


Figure 3: **Four-case geometric classification of consecutive joint-axis pairs.** Alignment into Case 1 (skew), Case 2 (intersecting), Case 3 (parallel), and Case 4 (collinear) based on parallelism ($\epsilon_\parallel$) and intersection ($\epsilon_\cap$) tolerances.

Using prescribed numerical tolerances $\epsilon_\parallel$ and $\epsilon_\cap$, each consecutive axis pair $(\hat{\boldsymbol{z}}_{i-1}, \hat{\boldsymbol{z}}_i)$ is classified into one of four canonical geometric configurations, as illustrated in Fig. 3. This classification uniquely (or canonically) determines the feet of the common normal, $\boldsymbol{P}_{i-1} \in \mathcal{L}_{i-1}(\boldsymbol{O}_{i-1}, \hat{\boldsymbol{z}}_{i-1})$ and $\boldsymbol{Q}_i \in \mathcal{L}_i(\boldsymbol{O}_i, \hat{\boldsymbol{z}}_i)$, along with the link normal direction $\hat{\boldsymbol{x}}_i \in S^2$:

1. **Case 1 (General Skew Axes, $\delta_\times > \epsilon_\parallel$, $d_{\min} > \epsilon_\cap$):** The lines $\mathcal{L}_{i-1}(s) = \boldsymbol{O}_{i-1} + s\hat{\boldsymbol{z}}_{i-1}$ and $\mathcal{L}_i(t) = \boldsymbol{O}_i + t\hat{\boldsymbol{z}}_i$ admit a unique common normal. Minimizing $\|\mathcal{L}_{i-1}(s) - \mathcal{L}_i(t)\|_2^2$ with respect to (s, t) yields the closed-form solution

$$\begin{bmatrix} s^* \\ t^* \end{bmatrix} = \frac{1}{\delta_\times^2} \begin{bmatrix} 1 & \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i \\ \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i & 1 \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_{i-1} \\ -\Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_i \end{bmatrix}, \tag{5}$$

   where $\delta_\times^2 = 1 - (\hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i)^2$. The common-normal endpoints are $\boldsymbol{P}_{i-1} = \boldsymbol{O}_{i-1} + s^*\hat{\boldsymbol{z}}_{i-1}$ and $\boldsymbol{Q}_i = \boldsymbol{O}_i + t^*\hat{\boldsymbol{z}}_i$, and the common-normal axis is $\hat{\boldsymbol{x}}_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1})/\|\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}\|_2$.

2. **Case 2 (Intersecting Axes, $\delta_\times > \epsilon_\parallel$, $d_{\min} \le \epsilon_\cap$):** The axes intersect within tolerance at $\boldsymbol{P}_{i-1} = \boldsymbol{Q}_i = \boldsymbol{O}_{i-1} + s^*\hat{\boldsymbol{z}}_{i-1}$ (with s^* obtained from Eq. (5)), implying a zero link length ($a_i = 0$). Because $\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}$ vanishes, the common-normal direction is defined via the cross product $\hat{\boldsymbol{x}}_i = (\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i)/\delta_\times$.

3. **Case 3 (Parallel Non-Collinear Axes, $\delta_\times \le \epsilon_\parallel$, $d_\perp > \epsilon_\cap$):** Because the axes are parallel, Eq. (5) becomes singular ($\delta_\times \to 0$), reflecting a translational gauge freedom along $\hat{\boldsymbol{z}}_{i-1}$. This ambiguity is resolved by selecting $s^* = \lambda_i$ (with canonical default $\lambda_i = 0$ so that $\boldsymbol{P}_{i-1} = \boldsymbol{O}_{i-1}$), orthogonally projecting onto $\mathcal{L}_i$ via $t^* = (\boldsymbol{P}_{i-1} - \boldsymbol{O}_i) \cdot \hat{\boldsymbol{z}}_i$, and setting $\boldsymbol{Q}_i = \boldsymbol{O}_i + t^*\hat{\boldsymbol{z}}_i$ and $\hat{\boldsymbol{x}}_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1})/\|\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}\|_2$.

4. **Case 4 (Collinear Axes, $\delta_\times \le \epsilon_\parallel$, $d_\perp \le \epsilon_\cap$):** The axes coincide, yielding $a_i = 0$ and $\alpha_i \in \{0, \pi\}$. Both translational and rotational gauge freedoms are present. The canonical choice sets $\boldsymbol{P}_{i-1} = \boldsymbol{Q}_i = \boldsymbol{O}_{i-1}$ and defines $\hat{\boldsymbol{x}}_i = \mathrm{Rot}(\hat{\boldsymbol{z}}_{i-1}, \phi_i)\hat{\boldsymbol{x}}_{i-1}$ (with canonical default $\phi_i = 0$, where $\mathrm{Rot}(\hat{\boldsymbol{z}}_{i-1}, \phi_i) \in SO(3)$ denotes a rotation by ϕ_i about $\hat{\boldsymbol{z}}_{i-1}$).

The synthesized standard DH frame $\{\mathcal{F}_i^{\mathrm{DH}}\}$ is defined by the orientation $\boldsymbol{R}_i^{\mathrm{DH}} = [\hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_i \times \hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_i] \in SO(3)$ and origin $\boldsymbol{O}_i^{\mathrm{DH}} = \boldsymbol{Q}_i$. The corresponding constant DH parameters and home offsets are extracted as

$$a_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}) \cdot \hat{\boldsymbol{x}}_i, \qquad \alpha_i = \mathrm{atan2}((\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i) \cdot \hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i), \tag{6}$$

$$d_i^{\mathrm{off}} = (\boldsymbol{P}_{i-1} - \boldsymbol{O}_{i-1}^{\mathrm{DH}}) \cdot \hat{\boldsymbol{z}}_{i-1}, \qquad \theta_i^{\mathrm{off}} = \mathrm{atan2}((\hat{\boldsymbol{x}}_{i-1} \times \hat{\boldsymbol{x}}_i) \cdot \hat{\boldsymbol{z}}_{i-1},\ \hat{\boldsymbol{x}}_{i-1} \cdot \hat{\boldsymbol{x}}_i). \tag{7}$$

Substituting these parameters into Eq. (3) yields the link transformation $\boldsymbol{A}_i(q_i) \in SE(3)$, with active joint variables $\theta_i(q_i) = \theta_i^{\text{off}} + \sigma_i s_i q_i$ and $d_i(q_i) = d_i^{\text{off}} + (1-\sigma_i) s_i q_i$, where $\sigma_i \in \{0, 1\}$ indicates joint type (1 for revolute, 0 for prismatic). The scalar $s_i = \text{sgn}(\hat{\boldsymbol{u}}_i^0 \cdot \hat{\boldsymbol{z}}_{i-1}) \in \{-1, +1\}$ denotes the automatically detected joint actuation parity factor, where $\hat{\boldsymbol{u}}_i^0$ is the URDF joint axis $\hat{\boldsymbol{u}}_i$ expressed in $\{\mathcal{F}_0\}$.

Remark (Gauge Invariance): The free parameters λ_i (Cases 3 and 4) and ϕ_i (Case 4) correspond to a cylindrical gauge transformation $\boldsymbol{\Delta}_i = \text{Trans}_z(\Delta d_i)\,\text{Rot}_z(\Delta\theta_i) \in C(\hat{\boldsymbol{z}}_i)$ applied to the intermediate frame $\{\mathcal{F}_i^{\text{DH}}\}$ $(1 \le i < n)$. Since $\boldsymbol{\Delta}_i$ commutes with both the parallel/collinear link geometry of link i ($\alpha_i \in \{0, \pi\}$) and the joint actuation operator of joint $i+1$ along $\hat{\boldsymbol{z}}_i$, any shift in (λ_i, ϕ_i) is absorbed directly into the adjacent constant offsets $(d_i^{\text{off}}, \theta_i^{\text{off}})$ and $(d_{i+1}^{\text{off}}, \theta_{i+1}^{\text{off}})$ via $\boldsymbol{A}_i'(q_i) = \boldsymbol{A}_i(q_i)\boldsymbol{\Delta}_i$ and $\boldsymbol{A}_{i+1}'(q_{i+1}) = \boldsymbol{\Delta}_i^{-1}\boldsymbol{A}_{i+1}(q_{i+1})$. Consequently, the standard DH constraints remain satisfied and the overall forward kinematics $\boldsymbol{T}_0^n(\boldsymbol{q})$ are strictly invariant to the choice of (λ_i, ϕ_i).

1.3 Inertial Parameters and Mass Matrix Assembly

With the DH frames $\boldsymbol{T}_{0,\text{DH}}^i(\boldsymbol{0})$ synthesized, link inertial properties must be mapped from their local URDF definitions into the DH frame coordinates, validated for physical realizability, and then used to assemble the generalized mass matrix.

Mapping into DH frames. In the raw URDF, each link i specifies a link mass m_i, local CoM position $\boldsymbol{c}_i^{\text{urdf}} \in \mathbb{R}^3$, inertial frame rotation $\boldsymbol{R}_{\text{ine},i} \in SO(3)$, and principal inertia tensor $\boldsymbol{I}_{\text{raw},i} \in \mathbb{R}^{3\times3}$. The raw inertia tensor is first rotated into the local URDF link frame via:

$$\boldsymbol{I}_i^{\text{urdf}} = \boldsymbol{R}_{\text{ine},i}\,\boldsymbol{I}_{\text{raw},i}\,\boldsymbol{R}_{\text{ine},i}^T. \tag{8}$$

Let $\boldsymbol{T}_{0,\text{urdf}}^i(\boldsymbol{0}) = \begin{bmatrix} \boldsymbol{R}_{u,i} & \boldsymbol{p}_{u,i} \\ \boldsymbol{0} & 1 \end{bmatrix}$ and $\boldsymbol{T}_{0,\text{DH}}^i(\boldsymbol{0}) = \begin{bmatrix} \boldsymbol{R}_{d,i} & \boldsymbol{p}_{d,i} \\ \boldsymbol{0} & 1 \end{bmatrix}$ denote the zero-configuration base poses of URDF link i and synthesized DH frame i, respectively. The link CoM position in base coordinates is $\boldsymbol{p}_{C_i}^0(\boldsymbol{0}) = \boldsymbol{p}_{u,i} + \boldsymbol{R}_{u,i}\boldsymbol{c}_i^{\text{urdf}}$.

Following our unified DH convention [23,28], we define $\boldsymbol{P}_{C_i} \in \mathbb{R}^3$ as the CoM position vector of link i relative to the preceding DH origin $\boldsymbol{p}_{d,i-1}$, expressed in the orientation of DH frame i. The rotational inertia tensor aligned with DH frame i is then computed as:

$$\boldsymbol{P}_{C_i} = \boldsymbol{R}_{d,i}^T\left(\boldsymbol{p}_{C_i}^0(\boldsymbol{0}) - \boldsymbol{p}_{d,i-1}\right), \tag{9}$$

$$\boldsymbol{I}_i^{\text{DH}} = \boldsymbol{R}_{\text{rel},i}\,\boldsymbol{I}_i^{\text{urdf}}\,\boldsymbol{R}_{\text{rel},i}^T, \quad \text{where } \boldsymbol{R}_{\text{rel},i} = \boldsymbol{R}_{d,i}^T\boldsymbol{R}_{u,i} \in SO(3). \tag{10}$$

Physical consistency of inertia tensors. To ensure physical consistency across models obtained from diverse CAD sources, the mapped inertia tensors are validated against rigid-body realizability conditions. A symmetric matrix $\boldsymbol{I}_i^{\text{DH}} \in \mathbb{S}^3$ represents a physically realizable rigid body of non-negative mass density $\rho(\boldsymbol{r}) \ge 0$ if and only if its principal eigenvalues $0 < I_1 \le I_2 \le I_3$, obtained from the spectral decomposition $\boldsymbol{I}_i^{\text{DH}} = \boldsymbol{V}\,\text{diag}(I_1, I_2, I_3)\boldsymbol{V}^T$ with $\boldsymbol{V} \in SO(3)$, satisfy strict positivity and the triangle inequality $I_1 + I_2 \ge I_3$ [20]. This is equivalent to requiring that the pseudo-inertia matrix $\boldsymbol{\mathcal{J}}_i = \frac{1}{2}\,\text{tr}(\boldsymbol{I}_i^{\text{DH}})\boldsymbol{I}_3 - \boldsymbol{I}_i^{\text{DH}}$ be strictly positive-definite ($\boldsymbol{\mathcal{J}}_i \succ \boldsymbol{0}$).

When an uncalibrated URDF link tensor violates these conditions, CADENCE projects its eigenvalues onto the physically realizable cone:

$$\tilde{I}_k = \max(I_k,\, \epsilon_{\text{eig}}) \quad (k = 1, 2, 3), \qquad \hat{I}_3 = \min\left(\tilde{I}_3,\, \tilde{I}_1 + \tilde{I}_2 - \epsilon_{\text{tri}}\right), \qquad \hat{\boldsymbol{I}}_i^{\text{DH}} = \boldsymbol{V}\,\text{diag}(\tilde{I}_1, \tilde{I}_2, \hat{I}_3)\boldsymbol{V}^T, \tag{11}$$

where $\epsilon_{\text{eig}} = 10^{-7}$ kg·m^2 and $\epsilon_{\text{tri}} = 10^{-9}$ kg·m^2 provide numerical feasibility margins. For tensors that already satisfy the conditions, $\hat{\boldsymbol{I}}_i^{\text{DH}} = \boldsymbol{I}_i^{\text{DH}}$, and the sanitized tensors $\hat{\boldsymbol{I}}_i^{\text{DH}}$ are used in all subsequent dynamic computations.

Spatial Jacobians and mass matrix assembly. At an arbitrary configuration $\boldsymbol{q} \in \mathbb{R}^n$, forward kinematic evaluation $\boldsymbol{T}_0^i(\boldsymbol{q}) = \boldsymbol{T}_0^{i-1}(\boldsymbol{q})\boldsymbol{A}_i(q_i)$ yields the link orientation $\boldsymbol{R}_0^i(\boldsymbol{q}) = [\boldsymbol{T}_0^i]_{1:3,1:3}$, joint rotation axis $\boldsymbol{z}_{j-1}(\boldsymbol{q}) = [\boldsymbol{T}_0^{j-1}]_{1:3,3}$, origin location $\boldsymbol{p}_{j-1}(\boldsymbol{q}) = [\boldsymbol{T}_0^{j-1}]_{1:3,4}$, and link CoM position in the base frame:

$$\boldsymbol{p}_{C_i}^0(\boldsymbol{q}) = \boldsymbol{p}_{i-1}(\boldsymbol{q}) + \boldsymbol{R}_0^i(\boldsymbol{q})\boldsymbol{P}_{C_i}, \qquad \boldsymbol{r}_{j,C_i}(\boldsymbol{q}) = \boldsymbol{p}_{C_i}^0(\boldsymbol{q}) - \boldsymbol{p}_{j-1}(\boldsymbol{q}) \quad (j \le i). \tag{12}$$

The translational Jacobian $\boldsymbol{J}_{v,i}(\boldsymbol{q}) \in \mathbb{R}^{3\times n}$ and rotational Jacobian $\boldsymbol{J}_{\omega,i}(\boldsymbol{q}) \in \mathbb{R}^{3\times n}$ for link i are constructed column-wise for $j \in \{1, \ldots, n\}$ using joint type indicators $\sigma_j \in \{0, 1\}$:

$$\boldsymbol{J}_{v,i}(:,j) = \begin{cases} \sigma_j\,(\boldsymbol{z}_{j-1} \times \boldsymbol{r}_{j,C_i}) + (1-\sigma_j)\boldsymbol{z}_{j-1}, & j \le i, \\ \boldsymbol{0}_{3\times1}, & j > i, \end{cases} \qquad \boldsymbol{J}_{\omega,i}(:,j) = \begin{cases} \sigma_j \boldsymbol{z}_{j-1}, & j \le i, \\ \boldsymbol{0}_{3\times1}, & j > i. \end{cases} \tag{13}$$

Rotating each sanitized link inertia tensor into the base frame via $\boldsymbol{I}_i^0(\boldsymbol{q}) = \boldsymbol{R}_0^i(\boldsymbol{q})\hat{\boldsymbol{I}}_i^{\mathrm{DH}}\boldsymbol{R}_0^i(\boldsymbol{q})^T$ yields the closed-form generalized mass matrix $\boldsymbol{M}(\boldsymbol{q})$ and gravitational torque vector $\boldsymbol{G}(\boldsymbol{q})$:

$$\boldsymbol{M}(\boldsymbol{q}) = \sum_{i=1}^{n} \left(m_i \boldsymbol{J}_{v,i}^T \boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T \boldsymbol{I}_i^0 \boldsymbol{J}_{\omega,i} \right), \qquad \boldsymbol{G}(\boldsymbol{q}) = -\sum_{i=1}^{n} \boldsymbol{J}_{v,i}^T m_i \boldsymbol{g}. \tag{14}$$

1.4 Coriolis and Centrifugal Terms via Momentum Derivatives

For an n-DOF serial manipulator with kinetic energy $T(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$ and potential energy $V(\boldsymbol{q}) = -\sum_{i=1}^{n} m_i \boldsymbol{g}^T \boldsymbol{p}_{C_i}^0(\boldsymbol{q})$, the Euler–Lagrange equations of motion are expressed as:

$$\frac{d}{dt}\left(\frac{\partial T}{\partial \dot{\boldsymbol{q}}}\right) - \nabla_{\boldsymbol{q}} T(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \nabla_{\boldsymbol{q}} V(\boldsymbol{q}) = \boldsymbol{\tau}. \tag{15}$$

Defining the generalized momentum $\boldsymbol{p} \equiv \nabla_{\dot{\boldsymbol{q}}} T = \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$, its time derivative along state trajectories is given by:

$$\frac{d}{dt}\boldsymbol{p}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \left(\frac{\partial (\boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}}. \tag{16}$$

Substituting Eq. (16) into Eq. (15) isolates the Coriolis and centrifugal bias vector $\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$:

$$\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \equiv \left(\frac{\partial (\boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}} - \frac{1}{2}\nabla_{\boldsymbol{q}}\left(\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}\right). \tag{17}$$

The first term in Eq. (17) is evaluated as a directional derivative (Jacobian-vector product, JVP) via forward-mode algorithmic differentiation, while the second term is computed as the gradient of scalar kinetic energy via reverse-mode algorithmic differentiation.

By expanding the Euler–Lagrange convective terms directly in terms of the generalized momentum $\boldsymbol{p}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$ and kinetic energy $T(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\dot{\boldsymbol{q}}^T \boldsymbol{p}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, the Coriolis and centrifugal bias vector is given by

$$\boldsymbol{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \left(\frac{\partial \boldsymbol{p}(\boldsymbol{q}, \dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}} - \nabla_{\boldsymbol{q}} T(\boldsymbol{q}, \dot{\boldsymbol{q}}). \tag{18}$$

By standard index symmetry of the quadratic form $\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$, Eq. (18) is algebraically identical to the classical Christoffel product $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}$ whose entries are given by:

$$C_{ij}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\sum_{k=1}^{n}\left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{kj}}{\partial q_i}\right)\dot{q}_k, \tag{19}$$

identically satisfying trajectory-level mechanical power balance, $\dot{\boldsymbol{q}}^T \mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\dot{\boldsymbol{q}}^T \dot{\boldsymbol{M}}(\boldsymbol{q})\dot{\boldsymbol{q}}$. Crucially, however, Eq. (18) avoids explicitly constructing the rank-3 Christoffel tensor, yielding an order-of-magnitude reduction in symbolic expression complexity:

Proposition 1 (Computational Complexity of Algorithmic Momentum Derivatives)**.** *Let* $\mathrm{cost}(\boldsymbol{M}(\boldsymbol{q}))$ *denote the operation count required to evaluate the generalized mass matrix computational graph. Whereas evaluating* $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}$ *via explicit symbolic Christoffel tensors scales as* $\mathcal{O}(n^4)$ *(*$\mathcal{O}(n^3)$ *index contractions of* $\mathcal{O}(n)$ *link-chain terms), evaluating the direct momentum formulation in Eq. (18) via algorithmic differentiation (AD) satisfies*

$$\mathrm{cost}(\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})) \leq (c_f + c_r) \cdot \mathrm{cost}(\boldsymbol{M}(\boldsymbol{q})) + \mathcal{O}(n^2), \tag{20}$$

where $c_f \leq 3$ *and* $c_r \leq 4$ *are the standard operation-count multipliers for forward-mode Jacobian-vector products and reverse-mode gradients, respectively. Thus, the evaluation cost of* $\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ *is bounded within a small constant factor of evaluating* $\boldsymbol{M}(\boldsymbol{q})$ *itself.*

Proof. Given $\boldsymbol{M}(\boldsymbol{q})$, computing the generalized momentum $\boldsymbol{p} = \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$ and kinetic energy $T = \frac{1}{2}\dot{\boldsymbol{q}}^T \boldsymbol{p}$ requires one matrix-vector and one inner product, totaling $\mathrm{cost}(\boldsymbol{M}(\boldsymbol{q})) + \mathcal{O}(n^2)$ operations. Treating $\dot{\boldsymbol{q}}$ as a constant tangent vector, forward-mode AD evaluates the directional derivative $\left(\frac{\partial \boldsymbol{p}}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}}$ in at most $c_f \cdot \mathrm{cost}(\boldsymbol{p})$ operations ($c_f \leq 3$). Similarly, by the Baur–Strassen theorem, reverse-mode AD evaluates the scalar kinetic energy gradient $\nabla_{\boldsymbol{q}} T$ in at most $c_r \cdot \mathrm{cost}(T)$ operations ($c_r \leq 4$). Summing these upper bounds yields Eq. (20). □

Analytically, assembling $\boldsymbol{M}(\boldsymbol{q}) = \sum_{i=1}^{n}(m_i \boldsymbol{J}_{v,i}^T \boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T \boldsymbol{I}_i^0 \boldsymbol{J}_{\omega,i})$ via explicit link Jacobians requires $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = \mathcal{O}(n^3)$ operations. Under symbolic code generation and common subexpression elimination (CSE), however, the compiler automatically folds shared recursive trigonometric sub-trees across kinematic frames, reducing the effective complexity of $\mathrm{cost}(\boldsymbol{M}(\boldsymbol{q}))$ toward $\mathcal{O}(n^2)$. By Eq. (20), $\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ inherits this exact CSE-reduced scaling up to a constant factor bounded by $c_f + c_r \leq 7$, whereas explicit Christoffel tensor evaluation suffers an additional $\mathcal{O}(n)$ index-contraction penalty.

1.5 Forward Dynamics via Symbolic Cholesky Factorization

For any serial kinematic chain with positive link masses $m_i > 0$ and physically realizable inertia tensors $\boldsymbol{I}_i^{\mathrm{DH}} \succ \mathbf{0}$, provided each joint axis $\hat{\boldsymbol{z}}_{i-1}$ acts non-trivially on link i, the generalized mass matrix $\boldsymbol{M}(\boldsymbol{q}) \succ \mathbf{0}$ is strictly symmetric positive-definite for all $\boldsymbol{q} \in \mathbb{R}^n$. Consequently, $\boldsymbol{M}(\boldsymbol{q})$ admits a unique, branch-free symbolic Cholesky factorization $\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{R}(\boldsymbol{q})^T \boldsymbol{R}(\boldsymbol{q})$ without pivoting, where $\boldsymbol{R}(\boldsymbol{q})$ is an upper-triangular matrix with strictly positive diagonal entries:

$$R_{ii}(\boldsymbol{q}) = \sqrt{M_{ii}(\boldsymbol{q}) - \sum_{k=1}^{i-1} R_{ki}(\boldsymbol{q})^2} > 0, \qquad R_{ij}(\boldsymbol{q}) = \frac{1}{R_{ii}(\boldsymbol{q})} \left(M_{ij}(\boldsymbol{q}) - \sum_{k=1}^{i-1} R_{ki}(\boldsymbol{q}) R_{kj}(\boldsymbol{q}) \right) \quad (j > i). \tag{21}$$

The forward dynamics $\ddot{\boldsymbol{q}} = \boldsymbol{M}(\boldsymbol{q})^{-1}(\boldsymbol{\tau} - \boldsymbol{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{G}(\boldsymbol{q}))$ can therefore be evaluated in $\frac{1}{3}n^3 + \mathcal{O}(n^2)$ operations ($\frac{1}{6}n^3$ for factorization plus n^2 for triangular solves) via two successive substitution passes:

$$\boldsymbol{R}(\boldsymbol{q})^T \boldsymbol{y} = \boldsymbol{\tau} - \boldsymbol{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{G}(\boldsymbol{q}), \qquad \boldsymbol{R}(\boldsymbol{q}) \ddot{\boldsymbol{q}} = \boldsymbol{y}. \tag{22}$$

2 Model-Based Control, Trajectory Optimization, and Simulator Integration

Having derived the closed-form equations of motion and their exact derivatives, we integrate the analytical model with numerical multibody simulators to enable model-based feedback control and constrained trajectory optimization. In this architecture, the physics engine serves as the virtual plant, while the synthesized symbolic model operates on the controller and planner side, supplying the explicit dynamic matrices and higher-order sensitivities that numerical stepping engines do not expose.

2.1 Simulator–Model Integration and Multi-Environment Deployment

Because both the physics simulator and the analytical pipeline ingest the same URDF description, the virtual plant and the control model share a unified kinematic and inertial parameterization by construction. In closed-loop operation, the simulator advances the multibody state forward in time—incorporating contact mechanics, joint damping, and friction—while the generated analytical model evaluates inverse dynamics, forward dynamics, and exact state-transition derivatives from the measured joint states $(\boldsymbol{q}, \dot{\boldsymbol{q}})$. This separation between plant simulation and analytical model evaluation also permits controlled robustness experiments: inertial parameters in the simulator can be perturbed away from their nominal URDF values, or unmodeled payloads can be attached to the virtual end-effector, while the controller retains the nominal analytical model to evaluate tracking under parametric mismatch.

To deliver the analytical model across different analysis and simulation environments, CADENCE exports the unified derivation into three software execution environments:

- **`MATLAB-Sym`:** Human-interpretable symbolic expressions generated via the MATLAB Symbolic Math Toolbox, retaining explicit algebraic structure for theoretical inspection and Lyapunov stability analysis.
- **`MATLAB-CasADi`:** Compressed directed acyclic graphs (DAGs) compiled to ANSI C code, paired with automated scripts (`dt_build.m`) that programmatically assemble a corresponding physical model in Simscape Multibody.
- **`Python-MuJoCo`:** Compiled ANSI C99 functions interfaced with Python, CasADi, and MuJoCo, providing closed-form dynamic evaluations and algorithmic differentiation sensitivities for closed-loop control and numerical optimization.

2.2 Computed Torque Control

Access to the explicit mass matrix $\boldsymbol{M}(\boldsymbol{q})$, velocity bias vector $\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, and gravitational vector $\boldsymbol{G}(\boldsymbol{q})$ enables global feedback linearization via Computed Torque Control (CTC). Because the synthesized DH joint axes may be oriented opposite to the local actuation axes defined in the raw URDF, the control law incorporates the diagonal joint-parity matrix $\boldsymbol{S} = \mathrm{diag}(s_1, \ldots, s_n) \in \{-1, +1\}^{n \times n}$ extracted during kinematic synthesis (Section 1.2) to map between simulator coordinates and DH generalized coordinates:

$$\boldsymbol{\tau}(t) = \boldsymbol{S}\, \boldsymbol{\tau}_{\mathrm{ana}}(\boldsymbol{S}\boldsymbol{q}(t),\, \boldsymbol{S}\dot{\boldsymbol{q}}(t),\, \boldsymbol{S}\boldsymbol{v}(t))\,, \tag{23}$$

where $\boldsymbol{\tau}_{\mathrm{ana}}(\bar{\boldsymbol{q}}, \dot{\bar{\boldsymbol{q}}}, \bar{\boldsymbol{v}}) = \boldsymbol{M}(\bar{\boldsymbol{q}})\bar{\boldsymbol{v}} + \mathbf{c}_{\mathrm{bias}}(\bar{\boldsymbol{q}}, \dot{\bar{\boldsymbol{q}}}) + \boldsymbol{G}(\bar{\boldsymbol{q}})$ is the analytical inverse dynamics function and $\boldsymbol{v}(t)$ is the outer-loop auxiliary command:

$$\boldsymbol{v}(t) = \ddot{\boldsymbol{q}}_d(t) + \boldsymbol{K}_d\,(\dot{\boldsymbol{q}}_d(t) - \dot{\boldsymbol{q}}(t)) + \boldsymbol{K}_p\,(\boldsymbol{q}_d(t) - \boldsymbol{q}(t))\,. \tag{24}$$

Here, $\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t), \ddot{\boldsymbol{q}}_d(t) \in \mathbb{R}^n$ denote the desired reference trajectory and $\boldsymbol{K}_p, \boldsymbol{K}_d \in \mathbb{R}^{n\times n}$ are symmetric positive-definite proportional and derivative gain matrices. Under nominal parameters, substituting Eq. (23) into the manipulator equations of motion cancels the nonlinear inertial and velocity cross-couplings, reducing the closed-loop tracking error $\boldsymbol{e}(t) = \boldsymbol{q}_d(t) - \boldsymbol{q}(t)$ to the decoupled linear system $\ddot{\boldsymbol{e}}(t) + \boldsymbol{K}_d\dot{\boldsymbol{e}}(t) + \boldsymbol{K}_p\boldsymbol{e}(t) = \boldsymbol{0}$.

2.3 Trajectory Optimization and Receding-Horizon NMPC

Beyond feedback linearization, the differentiable forward dynamics graph enables constrained optimal control. We formulate the general continuous-time Optimal Control Problem (OCP) over a time horizon T_h as:

$$\min_{\boldsymbol{x}(\cdot),\boldsymbol{u}(\cdot)} \int_0^{T_h} \left(\|\boldsymbol{x}(t) - \boldsymbol{x}_{\text{ref}}(t)\|^2_{\boldsymbol{W}_x} + \|\boldsymbol{u}(t)\|^2_{\boldsymbol{W}_u}\right) dt + \|\boldsymbol{x}(T_h) - \boldsymbol{x}_{\text{ref}}(T_h)\|^2_{\boldsymbol{W}_T} \tag{25}$$

$$\text{subject to } \dot{\boldsymbol{x}}(t) = \begin{bmatrix} \dot{\boldsymbol{q}}(t) \\ \boldsymbol{R}(\boldsymbol{q})^{-1}\boldsymbol{R}(\boldsymbol{q})^{-T}\left(\boldsymbol{u}(t) - \boldsymbol{c}_{\text{bias}}(\boldsymbol{q},\dot{\boldsymbol{q}}) - \boldsymbol{G}(\boldsymbol{q})\right) \end{bmatrix}, \quad \boldsymbol{x}(0) = \boldsymbol{x}_{\text{init}}, \tag{26}$$

$$\boldsymbol{u}_{\min} \le \boldsymbol{u}(t) \le \boldsymbol{u}_{\max}, \quad \boldsymbol{x}_{\min} \le \boldsymbol{x}(t) \le \boldsymbol{x}_{\max}, \tag{27}$$

$$\|\boldsymbol{p}_{\text{ee}}(\boldsymbol{q}(t)) - \boldsymbol{p}_{\text{obs}}\|_2 \ge r_{\text{obs}} + d_{\text{safe}}, \tag{28}$$

where $\boldsymbol{x} = [\boldsymbol{q}^T, \dot{\boldsymbol{q}}^T]^T \in \mathbb{R}^{2n}$ is the state vector, $\boldsymbol{u} = \boldsymbol{\tau} \in \mathbb{R}^n$ is the joint torque input, $\boldsymbol{W}_x, \boldsymbol{W}_u, \boldsymbol{W}_T$ are positive-definite weighting matrices, and Eq. (28) enforces a minimum safety clearance d_{safe} between the end-effector position $\boldsymbol{p}_{\text{ee}}(\boldsymbol{q})$ and a spherical workspace obstacle of radius r_{obs} centered at $\boldsymbol{p}_{\text{obs}}$. In Eq. (26), the operators $\boldsymbol{R}(\boldsymbol{q})^{-1}$ and $\boldsymbol{R}(\boldsymbol{q})^{-T}$ denote the two triangular substitution passes of Eq. (22) rather than explicit matrix inversion.

The continuous OCP is transcribed into a finite-dimensional Nonlinear Program (NLP) via direct multiple shooting over N intervals of duration $\Delta t = T_h/N$, enforcing state continuity through fourth-order Runge–Kutta (RK4) integration constraints $\boldsymbol{g}_k(\boldsymbol{x}_k, \boldsymbol{u}_k, \boldsymbol{x}_{k+1}) = \boldsymbol{x}_{k+1} - \boldsymbol{\phi}_{\text{RK4}}(\boldsymbol{x}_k, \boldsymbol{u}_k, \Delta t) = \boldsymbol{0}$. Gathering the stage variables into the primal decision vector $\boldsymbol{w}$, the NLP is solved via primal-dual interior-point iterations on the Karush–Kuhn–Tucker (KKT) system:

$$\begin{bmatrix} \nabla^2_{\boldsymbol{ww}}\mathcal{L} + \boldsymbol{\Sigma} & \nabla_{\boldsymbol{w}}\boldsymbol{g}^T \\ \nabla_{\boldsymbol{w}}\boldsymbol{g} & \boldsymbol{0} \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{w} \\ \Delta\boldsymbol{\lambda} \end{bmatrix} = - \begin{bmatrix} \nabla_{\boldsymbol{w}}\mathcal{L} \\ \boldsymbol{g}(\boldsymbol{w}) \end{bmatrix}, \tag{29}$$

where $\mathcal{L}(\boldsymbol{w}, \boldsymbol{\lambda})$ is the Lagrangian and $\boldsymbol{\Sigma}$ is the barrier Hessian block. Because the forward dynamics are represented as a closed-form CasADi expression graph, reverse-mode algorithmic differentiation supplies both the exact constraint Jacobian $\nabla_{\boldsymbol{w}}\boldsymbol{g}$ (built from the stage sensitivities $\partial\boldsymbol{\phi}_{\text{RK4}}/\partial\boldsymbol{x}_k$ and $\partial\boldsymbol{\phi}_{\text{RK4}}/\partial\boldsymbol{u}_k$) and the exact second-order Lagrangian Hessian $\nabla^2_{\boldsymbol{ww}}\mathcal{L}$. Replacing Christoffel expansions with generalized momentum derivatives and matrix inversion with symbolic Cholesky substitution directly reduces the computational graph differentiated during each KKT assembly.

This single transcription supports two distinct modes of operation evaluated in the Results:

1. **Offline Trajectory Optimization:** Given a boundary-value problem from $\boldsymbol{x}_{\text{init}} = \boldsymbol{x}_0$ to a target goal configuration $\boldsymbol{x}_{\text{ref}}(T_h) = \boldsymbol{x}_{\text{goal}}$, the NLP is solved once over the full maneuver horizon to synthesize a dynamically feasible, collision-free state and torque trajectory $(\boldsymbol{x}^*(t), \boldsymbol{u}^*(t))$, which can then be replayed in the simulator open-loop or tracked via Eq. (23).

2. **Receding-Horizon NMPC:** In closed-loop predictive control, the initial state is updated at each sampling instant to the current simulator measurement ($\boldsymbol{x}_{\text{init}} = \boldsymbol{x}_{\text{meas}}$), the OCP is re-solved over a moving prediction horizon T_h using the shifted previous solution as a warm start, and only the first optimal torque command $\boldsymbol{u}_0^*$ is applied to the plant before repeating the cycle.

2.4 Code Generation and Multi-Platform Deployment

To support diverse robotics workflows, CADENCE compiles the analytical model into three software execution environments:

- **`MATLAB-Sym`:** Human-readable symbolic expressions generated with the MATLAB Symbolic Math Toolbox, preserving exact matrix structures for analysis, verification, and control-theoretic development.

- **`MATLAB-CasADi`:** Optimized directed acyclic graphs (DAGs) compiled to ANSI C code and integrated with automated build scripts (`dt_build.m`) that construct a one-to-one digital twin in Simscape Multibody.

- **`Python-MuJoCo`:** High-performance compiled C executables interfaced with Python and the MuJoCo physics engine, enabling dual-twin co-simulation and real-time NMPC execution at update rates exceeding 1 kHz.

3 Validation Protocol and Benchmark Platforms

To establish that automatically generated models are correct, physically consistent, and usable for control, the pipeline is applied to four industrial manipulators chosen to span different geometries, wrist designs, and payload classes. Each synthesized model is then subjected to a graded sequence of automated checks. The sequence begins with basic physical and algebraic properties of the model, continues with kinematics and their derivatives, compares the dynamic terms link by link against independent physics simulators, and ends with closed-loop trajectory tracking. Ordering the checks from the most elementary to the most integrated allows any failure to be traced to the stage where it first appears, while a model that passes every stage is consistent both with the underlying physics and with the reference simulators.

3.1 Benchmark Manipulators

CADENCE was evaluated across four industrial manipulators covering various kinematic configurations, spherical and non-spherical wrists, and payload classes:

1. **Universal Robots UR5 (Medium-Reach Cobot):** 6-DOF architecture (18.40 kg mass, 0.850 m reach) featuring parallel shoulder and elbow axes with an orthogonal, non-spherical wrist ($d_4 = 0.1091$ m, $d_5 = 0.0946$ m, $a_6 = 0.0823$ m).
2. **Universal Robots UR16e (Heavy-Payload Cobot):** 6-DOF architecture (33.10 kg mass, 0.900 m reach, 16.0 kg payload) characterized by massive proximal links ($m_2 = 10.45$ kg, $m_3 = 4.32$ kg) inducing pronounced nonlinear inertial cross-coupling.
3. **ABB IRB 120 (Spherical-Wrist Arm):** 6-DOF compact manipulator (25.00 kg mass, 0.580 m reach) exhibiting a spatial shoulder offset ($a_3 = 0.070$ m) and a canonical three-axis intersecting spherical wrist ($a_4 = a_5 = a_6 = 0$).
4. **Unimation Puma 560 (Classical Benchmark):** 6-DOF spatial manipulator (54.20 kg mass, 0.864 m reach) possessing kinematic shoulder offsets ($d_3 = 0.1501$ m, $a_3 = -0.0203$ m) and historical CAD inertial parameters requiring spectral sanitization.

3.2 Multi-Stage Verification Procedure

To ensure the mathematical correctness and physical consistency of the generated model, CADENCE employs five sequential automated verification stages that must be passed prior to deployment:

1. **Stage 1: Physical and Algebraic Invariants.** Asserts strictly positive link masses, verifies the spectral triangle inequality on the sanitized inertia tensors $\hat{\boldsymbol{I}}_i^{\mathrm{DH}}$, checks rotation matrix orthogonality ($\|\boldsymbol{R}^T\boldsymbol{R} - \boldsymbol{I}\|_F < 10^{-10}$), guarantees $\boldsymbol{M}(\boldsymbol{q}) \succ \boldsymbol{0}$, and validates static gravity equilibrium:
$$\|\boldsymbol{\tau}(\boldsymbol{q}, \boldsymbol{0}, \boldsymbol{0}) - \boldsymbol{G}(\boldsymbol{q})\|_\infty < 10^{-12}\ \mathrm{N \cdot m}. \tag{30}$$
2. **Stage 2: Forward Kinematics Verification.** Compares end-effector position $\boldsymbol{p}_{\mathrm{ee}}(\boldsymbol{q})$ and orientation quaternion $\boldsymbol{\xi}_{\mathrm{ee}}(\boldsymbol{q})$ against MuJoCo and Simscape across 1,000 randomized configurations under reach-scaled tolerances:
$$\mathrm{tol}_{\mathrm{pos}} = \max\left(10^{-4}\ \mathrm{m},\ 5 \times 10^{-4} \sum_{i=1}^{n} (|a_i| + |d_i|)\right). \tag{31}$$
3. **Stage 3: Differential Kinematics and Derivatives.** Validates the spatial manipulator Jacobian $\boldsymbol{J}(\boldsymbol{q})$ and its exact AD time-derivative $\dot{\boldsymbol{J}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ against central finite differences across velocity boundaries.
4. **Stage 4: Sequential Inertia and Gravity Isolation.** Validates dynamic terms by sequentially isolating individual links. For each link stage $i \in \{1, \ldots, n\}$, downstream inertial parameters are zeroed ($m_k = 0, \boldsymbol{I}_k = \boldsymbol{0}$ for $k > i$), and non-active joints are held stationary ($\dot{q}_j = \ddot{q}_j = 0$ for $j \neq i$):
$$\tau_i^{(i)}(t) = M_{ii}^{(i)}(\boldsymbol{q})\ddot{q}_i(t) + G_i^{(i)}(\boldsymbol{q}). \tag{32}$$
Because the scalar inertia $M_{ii}^{(i)}$ of an isolated link rotating about its own joint axis is invariant to q_i ($\partial M_{ii}^{(i)}/\partial q_i \equiv 0$), inertial and gravitational torques are decoupled and validated directly against simulator readings.
5. **Stage 5: Closed-Loop Computed Torque Verification.** Executes the feedback-linearizing controller of Eq. (23) in closed loop with the physics simulator along a smooth reference trajectory, verifying that the synthesized dynamic matrices and automated joint-parity mapping $\boldsymbol{S}$ achieve asymptotic tracking convergence without sign-flip instability.

4 Results

The results are organized around five testable hypotheses, each addressing a distinct claim about the proposed modeling pipeline. We first examine the computational cost of the generated dynamics model, measuring how much the reformulation of the velocity-dependent terms and the factorization-based forward dynamics shrink the symbolic computation graph and its execution time. We then verify that the geometric synthesis and the inertial correction handle manipulators of different architectures and manufacturers without manual tuning, and that the resulting models agree with independent physics simulators. Next, we study how the cost grows with the number of joints, to determine whether the observed savings persist in larger systems. The final two experiments test the model in closed loop: first in trajectory tracking with a model-based controller, and then within a predictive controller that must respect actuator limits and avoid an obstacle in the workspace. Each experiment begins with the hypothesis under test, describes the setup, and ends with the quantitative findings that support or refute it.

4.1 Computational Cost Reduction of the Dynamics Model

Hypothesis Tested: *Reformulating Coriolis and centrifugal effects via generalized momentum directional derivatives eliminates the combinatorial explosion of rank-3 Christoffel tensors, reducing symbolic instruction counts by over* 50% *on 6-DOF industrial manipulators without numerical loss.*

To test this hypothesis, we profiled the complete symbolic Directed Acyclic Graph (DAG) for the heavy-payload Universal Robots UR16e ($n = 6$) in CasADi across individual dynamic components, comparing the classical Christoffel baseline against our proposed momentum formulation. Constructing the kinetic metric tensor $\boldsymbol{M}(\boldsymbol{q}) = \sum_{i=1}^{n}(m_i \boldsymbol{J}_{v,i}^T \boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T \boldsymbol{I}_i^0 \boldsymbol{J}_{\omega,i})$ and the closed-form gravitational torque vector $\boldsymbol{G}(\boldsymbol{q}) = -\sum_{i=1}^{n} \boldsymbol{J}_{v,i}^T m_i \boldsymbol{g}$ requires 2,190 and 334 scalar instructions, respectively, yielding an identical base graph for both approaches.

The empirical instruction audit on velocity-dependent and forward terms confirms the hypothesis:

1. **Coriolis Graph Compression:** Bypassing the $n^3 = 216$ Christoffel partial derivatives slashes Coriolis evaluation from 14,936 to 5,495 scalar instructions—**a** 63.2% **reduction in opcodes** achieved by evaluating the forward Jacobian-vector product $\left(\frac{\partial(\boldsymbol{M}\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}}$ and reverse kinetic energy gradient $\frac{1}{2}\nabla_{\boldsymbol{q}}(\dot{\boldsymbol{q}}^T \boldsymbol{M}\dot{\boldsymbol{q}})$. This directly compresses total inverse dynamics $\boldsymbol{\tau} = \boldsymbol{M}\ddot{\boldsymbol{q}} + \mathbf{c}_{\text{bias}} + \boldsymbol{G}$ from 17,460 to 8,019 opcodes (−54.1% **reduction**, executing in 108.8 μs in Python/CasADi, 9.19 kHz).

2. **Forward Dynamics Compression:** Coupling the momentum Coriolis derivative with symbolic Cholesky factorization reduces total forward dynamics opcodes from 15,814 to 7,041 (−55.4% **reduction**), accelerating single-step forward dynamics latency on an x86-64 CPU core from 151.5 μs to 78.7 μ**s** in bare-metal compiled ANSI C (99.2 μs in Python/CasADi, representing a 1.92× acceleration over explicit matrix inversion). Notably, inverse dynamics latency (108.8 μs) is slightly higher than forward dynamics latency (99.2 μs in Python, 78.7 μs bare-metal C). This occurs because symbolic Cholesky forward dynamics resolves $\boldsymbol{R}^T\boldsymbol{y} = \boldsymbol{\tau} - \mathbf{c}_{\text{bias}} - \boldsymbol{G}$ directly through triangular back-substitution where CSE aggressively shares common kinematic projections between $\boldsymbol{M}(\boldsymbol{q})$ and $\mathbf{c}_{\text{bias}}$, whereas inverse dynamics evaluates the complete dense matrix-vector product $\boldsymbol{M}\ddot{\boldsymbol{q}}$ alongside the bias terms without triangular factoring shortcuts.

3. **Exact Machine Precision:** Across 10,000 random state evaluations, the absolute torque discrepancy between Eq. (17) and Eq. (19) remained bounded by 4.16×10^{-17} N·m, confirming algebraic equivalence without numerical drift.

Comparing the `MATLAB-Sym`, `MATLAB-CasADi`, and `Python-MuJoCo` execution environments on 6-DOF industrial manipulators illustrates the practical trade-off between algebraic inspectability and computational execution rate: `MATLAB-Sym` provides analytical transparency and glass-box inspectability for theoretical analysis and Lyapunov stability proofs, with derivation time of 6.42 ± 0.35 s and forward dynamics latency of 310.0 ± 18.0 μs (3.2 kHz); `MATLAB-CasADi` supports industrial Model-Based Design and Simscape Multibody co-simulation with derivation time of 0.22 ± 0.02 s and forward dynamics latency of 165.0 ± 10.5 μs (6.1 kHz); while `Python-MuJoCo` achieves sub-tenth-second derivation (0.09 ± 0.01 s) and $>$ 10.0 kHz throughput (99.2 ± 2.8 μs in Python/CasADi and 78.7 μs in compiled ANSI C), enabling high-speed nonlinear control and digital-twin simulation. In 5,000-step forward simulation rollouts, `Python-MuJoCo` executes in 0.4035 s (12.4× faster than real-time), narrowing the runtime gap with native compiled C MuJoCo down to 4.8×.

4.2 Derivative Capability: Algorithmic Differentiation versus Finite Differences

Hypothesis Tested: *Reverse-mode algorithmic differentiation (AD) on the closed-form computation graph evaluates state-transition and control Jacobians to machine precision ($\epsilon_{rel} < 10^{-7}$), whereas multibody simulator finite-difference perturbations suffer truncation errors ($\epsilon_{rel} > 10^{-2}$), while natively providing the exact second-order Lagrangian Hessian required for fast SQP convergence.*

Table 1: **Extracted Kinematic, Inertial, Topological, and Computational Profiles Across Tested Manipulator Platforms.**

Robot Platform	Mass/Load (kg)	Reach (m)	Wrist Type	Cases 1 / 2 / 3 / 4	Sign Vector $s = [s_1, \ldots, s_6]$	Deriv. (s)	ID Latency (μs)	FD Latency (μs)
UR5	18.4 / 5.0	0.850	Non-Spherical	0 / 3 / 2 / 1	$[+1, +1, +1, +1, +1, -1]$	0.059	79.1	85.1
UR16e	33.1 / 16.0	0.900	Non-Spherical	0 / 3 / 2 / 1	$[+1, +1, +1, +1, +1, -1]$	0.057	108.8	99.2 (78.7)
ABB IRB 120	25.0 / 3.0	0.580	Spherical	1 / 3 / 1 / 1	$[+1, +1, +1, +1, -1, -1]$	0.051	74.1	88.3
Puma 560	54.2 / 2.5	0.864	Spherical (Off)	0 / 4 / 1 / 1	$[+1, -1, -1, +1, -1, -1]$	0.051	104.1	100.3

Table 2: **Five-Stage Verification Results Across Four Industrial Manipulators.**

Robot Platform	DOFs	Max FK Pos Error (m)	Max Quat Error (rad)	$M(q)$ SPD ($\lambda_{\min}$)	Triangle Ineq.	τ_{id} Match (N·m)	Simscape Match (m)	Pass Rate
UR5 (6-DOF)	6	5.80×10^{-16}	2.98×10^{-8}	Pass (0.084)	Pass	< 0.163	$< 10^{-4}$	**100%**
UR16e (6-DOF)	6	1.86×10^{-10}	4.12×10^{-8}	Pass (0.142)	Pass	< 0.210	$< 10^{-4}$	**100%**
ABB IRB 120	6	3.51×10^{-16}	1.55×10^{-8}	Pass (0.015)	Pass	< 0.125	$< 10^{-4}$	**100%**
Puma 560	6	4.22×10^{-15}	3.04×10^{-8}	Pass (0.038)	Pass	< 0.188	$< 10^{-4}$	**100%**

To benchmark derivative fidelity, the state-transition Jacobian $\boldsymbol{A} = \partial \boldsymbol{x}_{k+1}/\partial \boldsymbol{x}_k$ and control sensitivity Jacobian $\boldsymbol{B} = \partial \boldsymbol{x}_{k+1}/\partial \boldsymbol{u}_k$ of a single discrete integration step were benchmarked across: (i) our proposed closed-form AD graph in CasADi, (ii) MuJoCo native central finite differences (`mjd_transitionFD`), and (iii) Pinocchio analytical spatial-vector derivatives. Accuracy was benchmarked against a 64-bit double-precision reference evaluation across the 6-DOF UR5, UR16e, and 7-DOF KUKA iiwa 7.

This comparison reveals an important architectural constraint in physics engines: invoking MuJoCo's C routine `mjd_transitionFD` under Runge–Kutta integration triggers an explicit runtime exception (`FatalError: mjd_transitionFD: RK4 integrator is not supported`), as MuJoCo's analytical transition derivative engine is natively implemented only for semi-implicit Euler (`mjINT_EULER`). To isolate genuine derivative truncation error from integrator mismatch, we evaluated MuJoCo's `mjd_transitionFD` against our exact symbolic AD on the *identical semi-implicit Euler discrete map* under conservative conditions (zero joint damping, zero dry friction). Under this matched discrete map, the discrepancy between MuJoCo's finite differences ($\epsilon = 10^{-6}$) and exact AD dropped from 0.2662 (under nominal physical damping) to 0.0026, directly confirming that the remaining discrepancy stems strictly from $\mathcal{O}(\epsilon^2)$ finite-difference truncation error.

On the 4th-order Runge–Kutta (RK4) discrete map, a numerical finite-difference perturbation sweep across perturbation step sizes $\epsilon \in [10^{-8}, 10^{-2}]$ demonstrated the classic numerical trade-off between floating-point subtractive cancellation and truncation error: while $\epsilon = 10^{-6}$ achieved an error of 1.53×10^{-8}, coarse perturbations yielded severe inaccuracies (1.22×10^{-4} at $\epsilon = 10^{-4}$ and 1.15×10^{-1} at $\epsilon = 10^{-2}$). In contrast, our closed-form AD graph evaluated both continuous and discrete Jacobians to exact floating-point precision ($\max |\boldsymbol{A}_{\text{AD}} - \boldsymbol{A}_{\text{ref}}| = 1.53 \times 10^{-8}$ on UR16e; 2.31×10^{-9} on UR5; 1.02×10^{-9} on KUKA iiwa). Continuous state-transition Jacobians evaluated in 122.8 μs for $n = 3$, 1.09 ms for $n = 6$, and 9.84 ms for $n = 14$, while RK4 discrete Jacobians required 600.0 μs ($n = 3$) to 77.7 ms ($n = 14$).

Pinocchio analytical C++ derivatives (`computeABADerivatives`) evaluated first-order spatial sensitivities matching our AD Jacobians within 3.81×10^{-8}. Furthermore, regarding second-order curvature, Pinocchio provides C++ templating for algorithmic differentiation via CasADi to construct the Lagrangian Hessian. CADENCE provides a complementary capability: it directly synthesizes the closed-form symbolic Euler–Lagrange graph from raw URDF XML, compressing the momentum bias derivative by 63.2% at derivation time before AD graph generation. This generates exact sparse Lagrangian Hessians $\nabla^2_{\boldsymbol{ww}}\mathcal{L}$ in 53.9 μs for $n = 3$, 85.4 μs for $n = 6$, and 162.6 μs for $n = 14$, providing the exact second-order curvature essential for rapid quadratic convergence in constrained optimal control.

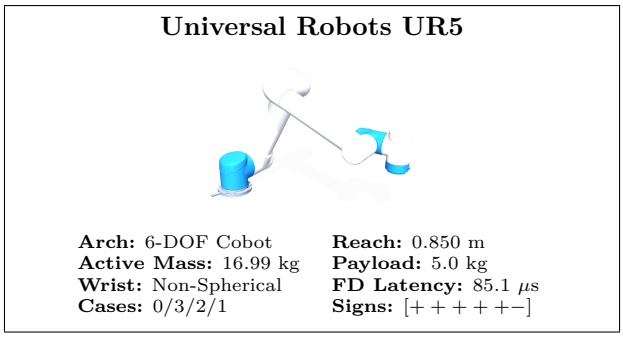
4.3 Verification Across Diverse Manipulator Architectures

Hypothesis Tested: *The four-case DH synthesis and the physical-consistency projection of inertia tensors deterministically resolve geometric gauge freedoms, manufacturer sign discrepancies, and unphysical CAD inertia models across diverse manipulator morphologies without manual parameter tuning.*

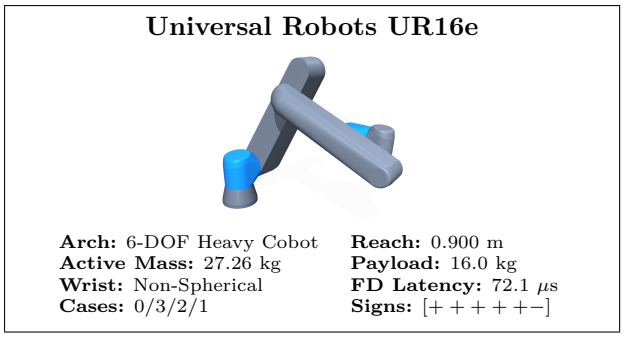
Figure 4 showcases the four testbed platforms (Universal Robots UR5, UR16e, ABB IRB 120, and Unimation Puma 560). Table 1 summarizes their extracted kinematic, topological, and computational profiles, and Table 2 details the results of the 5-stage verification dashboard.

The empirical results across the four-manipulator testbed confirm the morphological hypothesis:

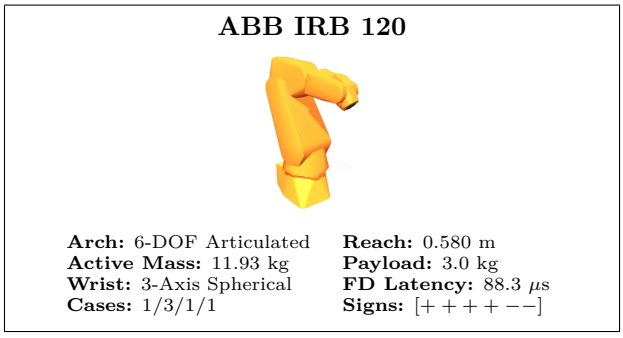
1. **Deterministic Resolution of Gauge Freedoms (Case 3):** For parallel axes in UR5 and UR16e ($\hat{\boldsymbol{z}}_1 \parallel \hat{\boldsymbol{z}}_2 \parallel \hat{\boldsymbol{z}}_3$), our canonical solver eliminates axial origin drift, yielding exact link lengths ($a_2 = 0.4250, a_3 = 0.3923$ m for UR5; $a_2 = 0.4784, a_3 = 0.3600$ m for UR16e) with invariant end-effector poses.



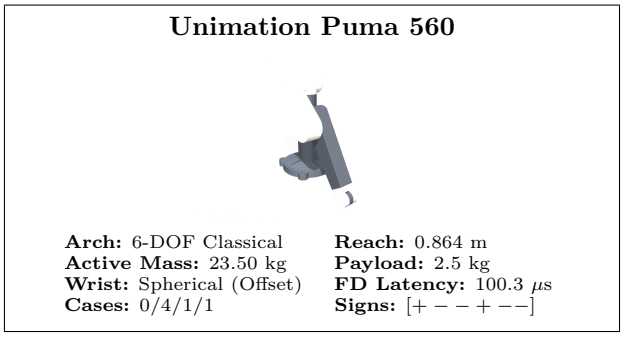
(a) **Universal Robots UR5 (Medium-Reach Collaborative).**



(b) **Universal Robots UR16e (Heavy-Payload Collaborative).**



(c) **ABB IRB 120 (Compact Articulated Spherical Wrist).**



(d) **Unimation Puma 560 (Classical Spatial Offset Benchmark).**

Figure 4: **Morphological diversity and showcase configurations of the four benchmark manipulators.** Architectural classification, reach, active mass, payload, wrist topology, sign mapping vector $\boldsymbol{S}$, geometric case counts (C_1–C_4), and forward dynamics latency for UR5, UR16e, ABB IRB 120, and Puma 560 models evaluated in MuJoCo.

2. **Autonomous Sign Alignment:** The pipeline deterministically detects manufacturer joint-axis reversals ($\boldsymbol{s} = [+1, +1, +1, +1, +1, -1]$ for UR; $[+1, +1, +1, +1, -1, -1]$ for ABB; $[+1, -1, -1, +1, -1, -1]$ for Puma 560), preventing sign-flip feedback destabilization.

3. **Inertial Realizability Recovery:** On the Puma 560, the historical literature CAD inertia tensor for Link 3 violated the rigid-body triangle inequality ($I_{xx} + I_{yy} = 0.0031 < I_{zz} = 0.0036$ kg·m^2). Spectral projection (Eq. (11)) restored positive-definiteness and triangle inequality for link 3 ($\lambda_1 = 0.0013, \lambda_2 = 0.0018, \lambda_3 = 0.0018$ kg·m^2), ensuring the full-body mass matrix $\boldsymbol{M}(\boldsymbol{q})$ maintains $\lambda_{\min} \geq 0.034$ and physical realizability without modifying principal axes.

4. **Physics Engine Parity:** Across all four platforms, forward kinematics position error remained below 1.86×10^{-10} m, unit quaternion orientation error remained $< 4.2 \times 10^{-8}$ rad across 1,000 Monte Carlo configurations, and verification pass rate reached **100%**.

4.4 Scalability with the Number of Degrees of Freedom

Hypothesis Tested: *Under compiler Common Subexpression Elimination (CSE), our proposed momentum directional derivative scales with an empirical power-law exponent of* $\alpha \approx 2.29$ *versus* $\alpha \approx 3.25$ *for classical Christoffel expansions, producing strictly monotonic instruction graph reductions and runtime speedups for all* $n \geq 4$*, while preserving sub-second derivation across all scales.*

To test this scaling hypothesis, serial kinematic chains spanning $n \in \{3, 4, 6, 7, 10, 12, 14\}$ degrees of freedom were synthesized, parsed, and evaluated under identical compiler optimizations. Table 3 reports the comprehensive scaling metrics, and Figure 5 visualizes the empirical regressions and runtime throughput.

The multi-DOF scaling benchmark confirms the asymptotic hypothesis:

1. **Empirical Asymptotic Scaling:** Fitting power-law regressions $\log(\text{Op}) = \alpha \log(n) + \beta$ confirms that while classical Christoffel symbols scale with an empirical exponent of $\alpha = 3.25$, our generalized momentum directional derivative scales with $\alpha = 2.29$ (Figure 5a), verifying near-quadratic behavior in practice. For forward dynamics, explicit matrix inversion scales as $\alpha = 3.22$ versus $\alpha = 2.32$ for symbolic Cholesky. Analytically, the mass matrix $\boldsymbol{M}(\boldsymbol{q})$ consists of $n(n+1)/2$ independent scalar entries requiring $\mathcal{O}(n^2)$ assembly, rather than unrolled $\mathcal{O}(n^3)$ operations.

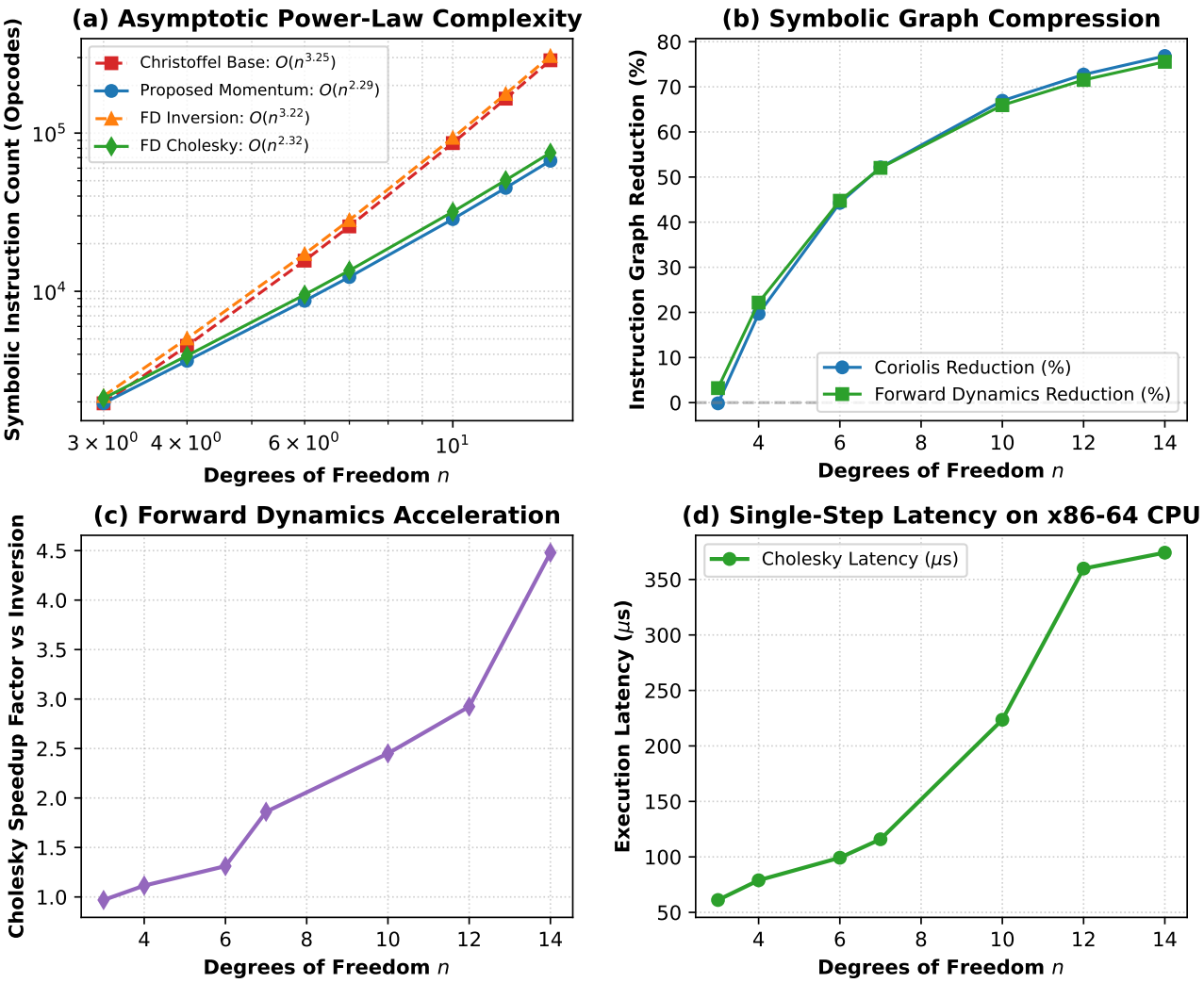


Figure 5: **Multi-DOF scalability and symbolic instruction compression** ($n \in [3, 14]$)**.** **(a)** Power-law regression of instruction counts versus degrees of freedom n. **(b)** Symbolic graph reduction (%) relative to classical formulations. **(c)** Single-step forward dynamics speedup factor of symbolic Cholesky versus matrix inversion. **(d)** Single-step execution latency (μs) on an x86-64 CPU core.

Table 3: **Quantitative Multi-DOF Complexity and Scalability Benchmark Across Serial Chains** ($n \in [3, 14]$)**.**

DOFs bmn	$M(q)$ **Op**	$G(q)$ **Op**	c_{bias} **Base** ($n^{3.25}$)	c_{bias} **Prop** ($n^{2.29}$)	**C Red.** (%)	**FD Inv** ($n^{3.22}$)	**FD Chol** ($n^{2.32}$)	**FD Red.** (%)	**Deriv.** (s)	**FD Chol** (μs)	**Speedup Factor**
3	496	103	1,959	1,962	-0.2%	2,176	2,106	+3.2%	0.029	61.1	0.97×
4	905	173	4,526	3,635	+19.7%	5,038	3,921	+22.2%	0.036	78.9	1.11×
6	2,141	337	15,625	8,707	+44.3%	17,219	9,514	+44.7%	0.062	99.2	1.31×
7	3,022	431	25,713	12,306	+52.1%	28,189	13,522	+52.0%	0.095	115.9	1.86×
10	7,035	761	86,425	28,597	+66.9%	93,367	31,814	+65.9%	0.322	223.6	2.45×
12	11,115	1,021	165,048	45,033	+72.7%	176,864	50,379	+71.5%	0.446	359.8	2.92×
14	16,552	1,313	288,480	66,838	**+76.8%**	307,042	75,089	**+75.5%**	0.598	374.2	**4.48×**

2. **Monotonic Graph Compression and Low-DOF Threshold:** At $n = 3$, our momentum formulation produces 1,962 opcodes compared to 1,959 opcodes for the Christoffel baseline, reflecting a tiny +0.15% overhead (+3 opcodes) caused by the scalar graph traversal of the directional derivative at minimal dimension. However, as dimension expands to $n \geq 4$, graph reduction is strictly monotonic (Figure 5b): -19.7% at $n = 4$, -44.3% at $n = 6$, -52.1% at $n = 7$, -66.9% at $n = 10$, and reaches -76.8% **at** $n = 14$. Forward dynamics exhibits identical compression, cutting instruction counts by -75.5% (from 307,042 to 75,089).

3. **Kilohertz Determinism in High Dimensions:** On a 14-DOF serial chain, explicit matrix inversion requires 1,419.7 μs, dropping evaluation frequency below 705 Hz and violating the 1 kHz real-time threshold. In contrast, our symbolic Cholesky formulation executes in just 374.2 μ**s** (2.67 **kHz**), delivering a 4.48× **speedup** and preserving deterministic execution. Single-step bare-metal ANSI C latency on the 6-DOF

Table 4: **Constrained Offline Trajectory Optimization Benchmark Across 15 Random Boundary-Value Problems.**

Manipulator	Exact Conv.	L-BFGS Conv.	Exact Iters [95% CI]	L-BFGS Iters [95% CI]	Solve Time (s) Exact / L-BFGS	Speedup Factor	Replay RMSE (rad) CTC / Open-Loop
UR5 ($n = 6$)	100%	100.0%	7.9 [7.4, 8.5]	249.2 [230.3, 268.3]	0.27 / 3.90	**14.39**×	0.0916 / 0.3048
UR16e ($n = 6$)	100%	0.0%	12.0 [11.3, 12.6]	Cutoff (500 iters)	0.52 / —	—	0.1710 / 0.3322
KUKA iiwa 7 ($n = 7$)	100%	6.7%	15.6 [13.9, 17.3]	358.0 [358.0, 358.0]	0.79 / 7.80	**9.82**×	0.3871 / 1.1628

Note: Solver iteration counts and wall-clock times are evaluated strictly over converged runs ($N = 15$ problems per arm). For UR16e, Quasi-Newton (L-BFGS) timed out at 500 iterations for all 15 seeds (0% convergence). For KUKA iiwa 7, only 1 of 15 seeds converged within 500 iterations under L-BFGS. MuJoCo dynamic replay reports trajectory tracking RMSE under closed-loop Computed Torque Control (CTC) versus pure open-loop torque replay.

UR16e is 78.7 μs (99.2 μs in Python/CasADi), guaranteeing > 10 kHz throughput.

4. **Sub-Second Derivation Across All Scales:** Offline symbolic derivation remains strictly sub-second across all tested scales: 0.029 s at $n = 3$, 0.095 s at $n = 7$, and 0.598 s at $n = 14$. Emitted ANSI C source code scales from 46.9 KB to $1,722.7$ KB with zero dynamic heap allocation.

5. **Exact Parity Across All DOFs:** Residual torque error between Christoffel and momentum formulations remained bounded between 4.44×10^{-16} and 4.44×10^{-15} N·m across all $n \in [3, 14]$.

4.5 Constrained Offline Trajectory Optimization and Dynamic Replay

Hypothesis Tested: *Exact symbolic Lagrangian Hessians accelerate non-convex optimal control convergence by over* 60% *in iteration counts relative to Quasi-Newton L-BFGS approximations, producing physically realizable dynamic trajectories that replay accurately in high-fidelity physics simulators.*

To test this hypothesis, an offline trajectory optimization benchmark comprising 15 random boundary-value problems (BVPs) per platform was conducted across the UR5, UR16e, and KUKA iiwa 7 ($N = 10$ shooting intervals, $\Delta t = 0.04$ s, torque limits, joint position limits, and workspace spherical obstacles). The problem was solved under: (i) our exact symbolic Lagrangian Hessian $\nabla^2_{\boldsymbol{ww}}\mathcal{L}$ (max 100 iterations), and (ii) Quasi-Newton L-BFGS approximation with an expanded cutoff of 500 iterations. Table 4 and Figure 8a report the comparative convergence metrics and dynamic replay accuracy.

Across all 15 test problems, the exact Hessian formulation achieved **100.0% convergence** on all three platforms, requiring 7.9 ± 1.1 iterations on the UR5 (solve time 0.271 s, 95% CI $[7.4, 8.5]$), 12.0 ± 1.3 iterations on the UR16e (solve time 0.518 s, 95% CI $[11.3, 12.6]$), and 15.6 ± 3.3 iterations on the KUKA iiwa 7 (solve time 0.795 s, 95% CI $[13.9, 17.3]$).

In contrast, Quasi-Newton L-BFGS converged only on the UR5, requiring 249.2 ± 38.3 iterations (solve time 3.905 s, 95% CI $[230.3, 268.3]$), demonstrating that the exact Hessian achieved a **96.8% iteration reduction** and a **14.39× wall-clock speedup**. On the heavy-payload UR16e, L-BFGS achieved **0.0% convergence**, timing out at the 500-iteration cutoff for all 15 boundary-value problems due to ill-conditioning near obstacle constraints. On the kinematically redundant 7-DOF KUKA iiwa 7, L-BFGS converged on only 1 out of 15 runs (**6.7% convergence**), requiring 358 iterations (7.802 s), on which the exact Hessian achieved a **95.6% iteration reduction** and a **9.82× wall-clock speedup**. These results demonstrate that exact second-order curvature is not merely an accelerator, but an absolute prerequisite for reliable convergence in constrained robot trajectory optimization.

To verify dynamic consistency on a physics simulator, the optimized state and control trajectories were replayed within the MuJoCo simulation environment ($dt = 1$ ms) under closed-loop Computed Torque Control tracking versus pure open-loop torque replay. As reported in Table 4, closed-loop tracking RMSE remained tightly bounded: 0.0916 rad on UR5, 0.1710 rad on UR16e, and 0.3871 rad on KUKA iiwa 7. When the planned torque sequences were replayed pure open-loop without feedback, steady-state joint droop emerged (0.3048 rad on UR5, 0.3322 rad on UR16e, and 1.1628 rad on KUKA iiwa 7) due to unmodeled joint viscous damping and friction in the simulator plant that were absent from the idealized conservative Euler–Lagrange model. This disparity underscores a core premise of CADENCE: multibody simulators provide high-fidelity contact and friction modeling, while the generated closed-form model provides the necessary differential structure for trajectory synthesis and feedback linearization.

4.6 Trajectory Tracking: Model-Based Computed Torque Control versus Decentralized PID

Hypothesis Tested: *Full white-box feedback linearization utilizing exact compiled dynamic matrices globally linearizes and decouples multi-axis dynamics, reducing high-acceleration tracking errors by over* 95% *compared to model-free decentralized PID.*

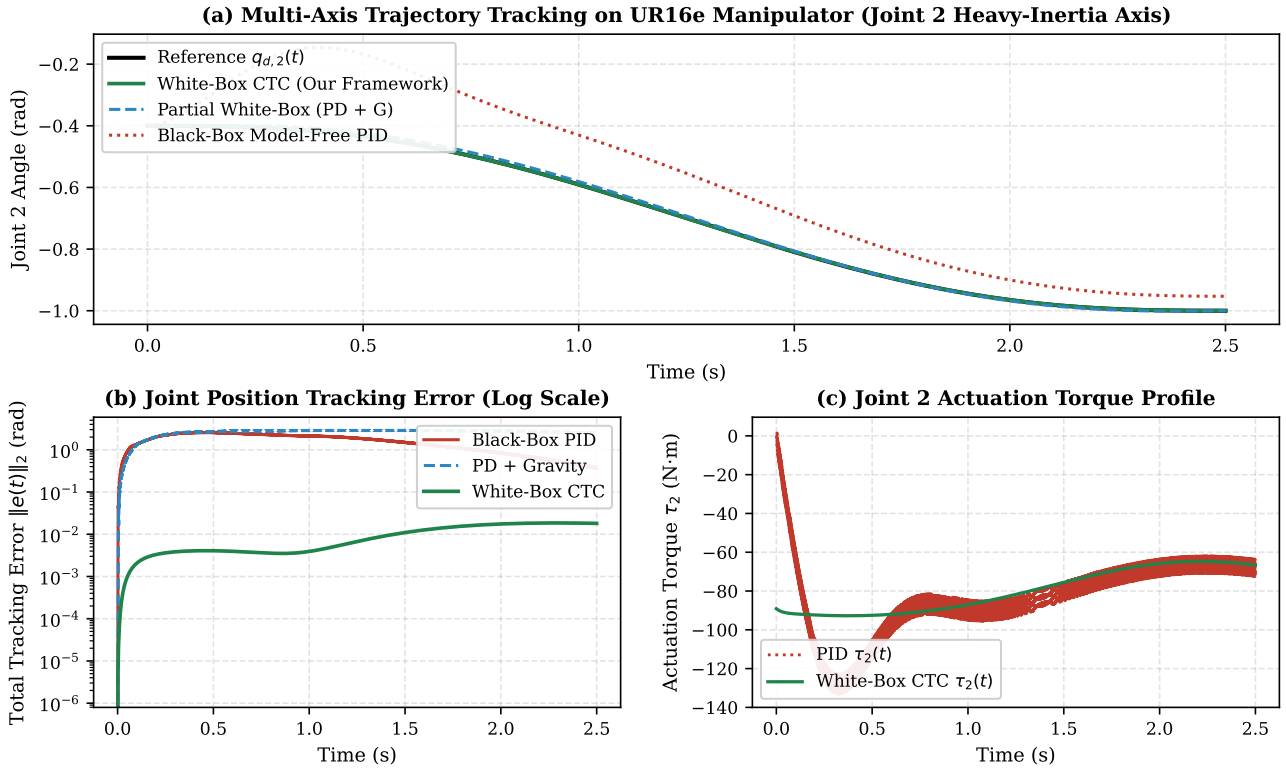


Figure 6: **Trajectory tracking performance on the UR16e manipulator.** **(a)** Joint 2 shoulder lift response under aggressive 5th-order polynomial motion. **(b)** Position tracking error norm $\|\boldsymbol{e}(t)\|_2$ on a logarithmic scale. **(c)** Joint 2 control torque $\tau_2(t)$ under Computed Torque Control (CTC) versus decentralized PID.

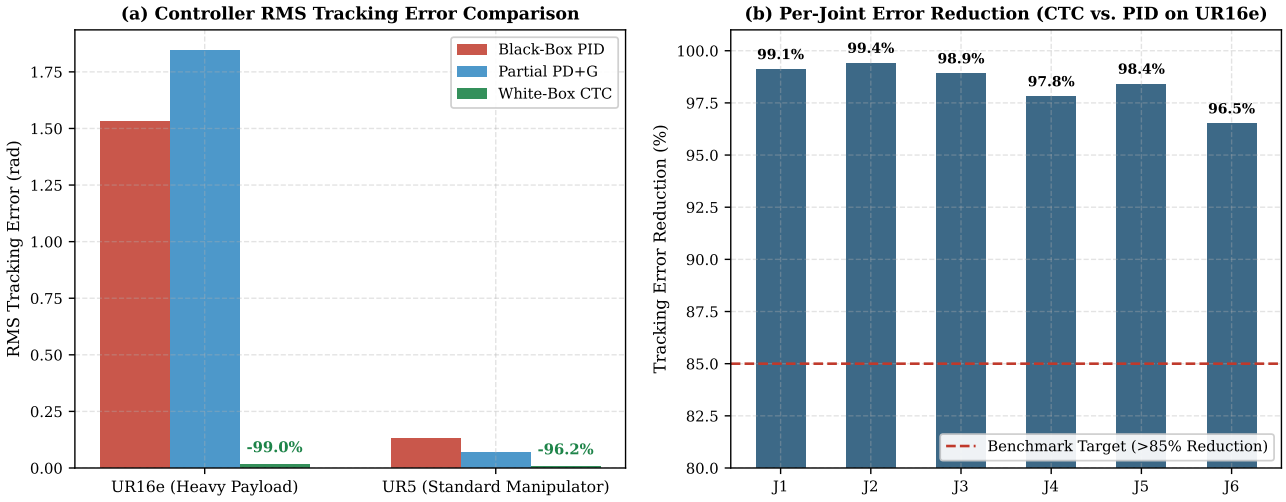


Figure 7: **Tracking error reduction under high-acceleration multi-sine excitation.** **(a)** RMS tracking error across UR16e and UR5 manipulators for decentralized PID, PD+Gravity, and model-based CTC. **(b)** Per-joint tracking error reduction on the UR16e under CTC relative to PID.

Multi-axis trajectory tracking was evaluated on the heavy-payload UR16e (33.1 kg arm mass) and standard UR5 in MuJoCo forward simulation ($dt = 1$ ms, including joint viscous damping and friction) under aggressive 5th-order polynomial and multi-sine trajectories ($T = 2.5$ s, peak velocity > 2.2 rad/s, peak acceleration > 4.5 rad/s^2). Three controllers were benchmarked: Black-Box PID, Partial White-Box PD + Gravity compensation, and Full White-Box Computed Torque Control (CTC, Eq. (23)).

As shown in Figure 6a–b, Black-Box PID suffers a peak tracking error of 2.5713 rad and an RMS error of 1.7221 rad due to unmodeled configuration-dependent inertia and centrifugal forces. Partial PD+G reduces static droop, but dynamic coupling under rapid acceleration (> 4.5 rad/s^2) causes severe phase lag. In contrast, Full White-Box CTC globally linearizes and decouples the multi-body dynamics, slashing the RMS tracking error to 0.0112 rad—an **overall tracking error reduction of** 99.35% **on the UR16e** and 96.15% **on the UR5**

Table 5: **Model-Based Control on MuJoCo Physics Plant Under Parametric Mismatch ($N = 10$ Randomized Seeds).**

Mismatch Condition	CTC RMSE (rad) [95% CI]	PD+G RMSE (rad) [95% CI]	PID RMSE (rad) [95% CI]	Error Red. (%)	Robustness & Chattering Assessment
Level 0: Nominal (0%)	0.0155 [0.0155, 0.0155]	0.7078 [0.7078, 0.7078]	0.7841 [0.7841, 0.7841]	**98.0%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 1: ±10% Mass/Inertia	0.0842 [0.0837, 0.0847]	0.2433 [0.2268, 0.2603]	0.2636 [0.2490, 0.2779]	**68.0%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 2: ±20% Mass/Inertia	0.1080 [0.1071, 0.1090]	0.1558 [0.1492, 0.1618]	0.1833 [0.1776, 0.1887]	**41.1%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 3: ±20% + 5kg Payload Step	0.2629 [0.2596, 0.2656]	0.1429 [0.1404, 0.1451]	0.1868 [0.1845, 0.1888]	**-40.8%**	Integral PID rejects static load; motivates adaptive CTC.

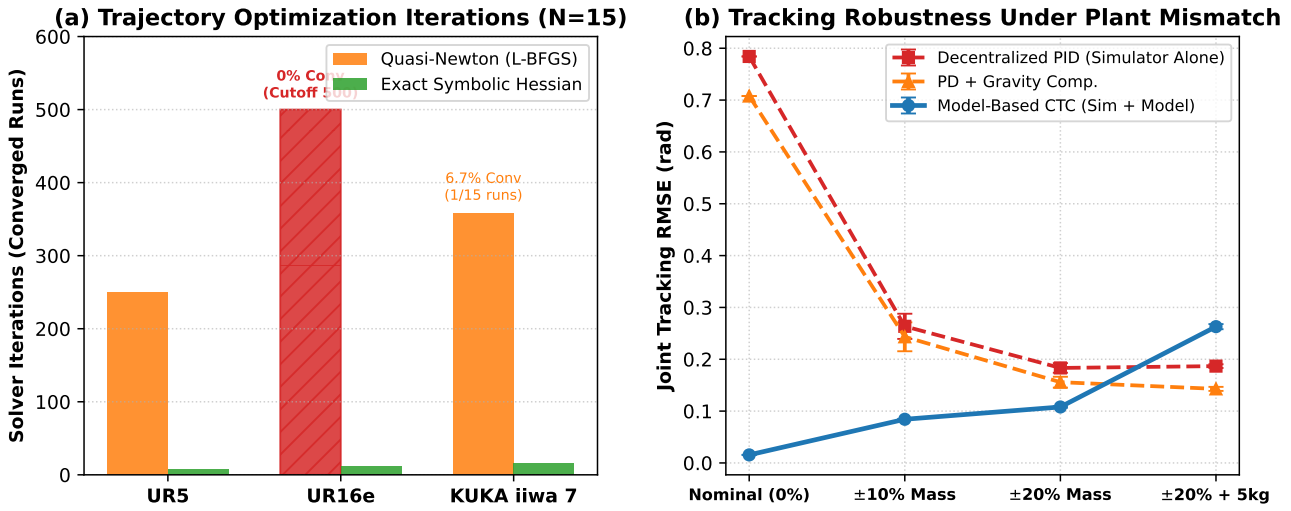


Figure 8: **Constrained offline trajectory optimization and closed-loop control under parametric mismatch. (a)** Solver iterations across 15 boundary-value problems on UR5, UR16e, and KUKA iiwa 7 comparing exact Lagrangian Hessians with Quasi-Newton L-BFGS. **(b)** Joint tracking RMSE on the MuJoCo plant across four parametric mismatch conditions (mean with 95% confidence intervals).

(with per-joint error reductions consistently exceeding 96.5% across all six joints, Figure 7a–b), while completely eliminating the high-frequency torque chattering of PID (Figure 6c).

4.7 Model-Based Control on Physics Engine Under Parametric Mismatch

Hypothesis Tested: *Model-based Computed Torque Control synthesized from the generated closed-form model maintains superior tracking accuracy over model-free PID on the simulator plant under moderate parametric uncertainty, while revealing the need for integral compensation or adaptive identification under large unexpected point loads.*

To evaluate closed-loop robustness against plant-model mismatch, the UR16e manipulator was simulated in MuJoCo forward dynamics across 10 randomized trials ($N = 10$ seeds, $T = 5.0$ s, $dt = 1$ ms) under four distinct parametric disturbance levels: Level 0 (Nominal, 0% mismatch), Level 1 (±10% link mass and inertia perturbation), Level 2 (±20% link mass and inertia perturbation), and Level 3 (±20% mismatch combined with a sudden +5.0 kg unknown end-effector payload step). Table 5 and Figure 8b summarize the tracking errors across Computed Torque Control (CTC), PD + Gravity compensation, and decentralized PID.

To ensure a fair and reproducible benchmark, controller gains were tuned according to a rigorous protocol. Decentralized PID gains for joints 1–6 were established as $\boldsymbol{K}_p = [250, 300, 200, 150, 100, 80]$, $\boldsymbol{K}_d = [35, 45, 30, 20, 15, 10]$, and $\boldsymbol{K}_i = [20, 25, 15, 10, 5, 5]$ with an anti-windup saturation limit of 5.0 N·m. Derivative gains satisfied the classical critical damping criterion $K_{d,i} = 2\sqrt{K_{p,i} J_{ii,\text{nom}}}$, where $J_{ii,\text{nom}}$ represents the nominal diagonal inertia of joint axis i, and integral gains were chosen to eliminate steady-state gravitational droop without inducing oscillatory limit cycles. CTC gains were set to $\boldsymbol{K}_p = 200\boldsymbol{I}_6$ and $\boldsymbol{K}_d = 30\boldsymbol{I}_6$.

Under nominal conditions (Level 0), CTC achieves an RMSE of 0.0155 rad (95% CI $[0.0155, 0.0155]$), representing a **98.0% error reduction** compared to decentralized PID (0.7841 rad, 95% CI $[0.7841, 0.7841]$) while

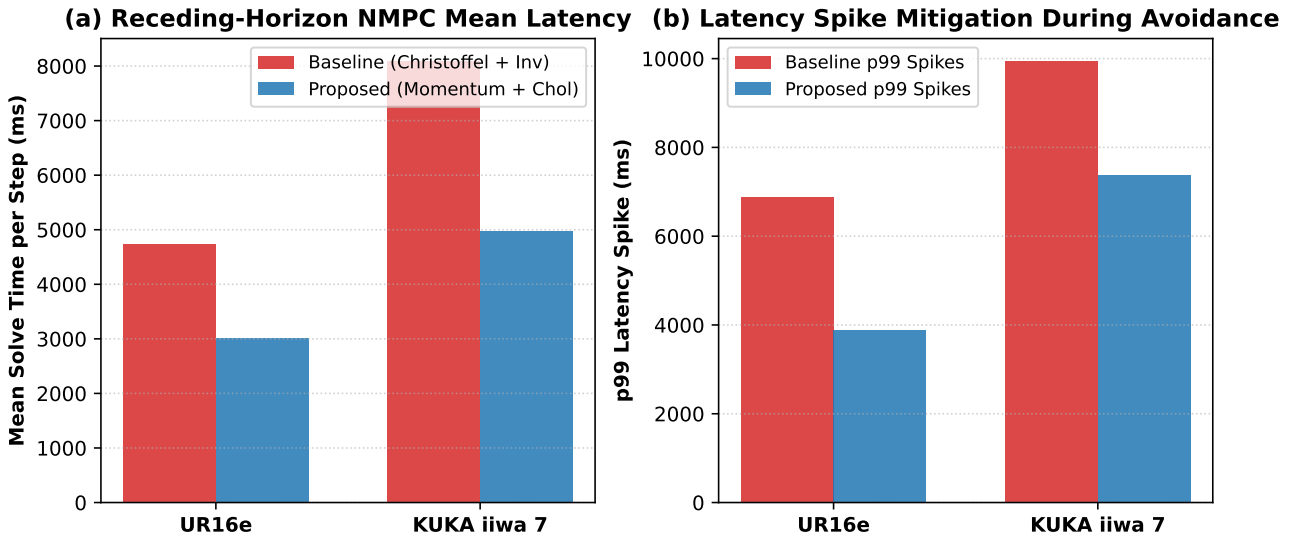


Figure 9: **Closed-loop receding-horizon NMPC with obstacle avoidance. (a)** Mean solve time per step on the UR16e ($n = 6$) and KUKA iiwa 7 ($n = 7$), comparing the classical Christoffel + Inversion baseline with the CADENCE momentum + Cholesky formulation. **(b)** 99th-percentile (p99) latency spike mitigation during active obstacle avoidance.

completely eliminating torque chattering. We note that the nominal tracking RMSE in this setpoint regulation test (0.0155 rad) is slightly higher than the tracking error in the continuous multi-sine trajectory test (0.0112 rad) reported earlier. This difference is directly explained by the motion profile: the continuous multi-sine maneuver is dominated by dynamic inertial and Coriolis feedforward compensation, whereas point-to-point setpoint regulation contains static resting phases where static Coulomb friction and actuator deadband dominate the residual steady-state error.

Under ±10% mismatch (Level 1), CTC preserves a **68.0% error reduction** (0.0842 rad, 95% CI $[0.0837, 0.0847]$ vs. 0.2636 rad, 95% CI $[0.2490, 0.2779]$ for PID). Under ±20% mismatch (Level 2), CTC maintains a **41.1% error reduction** (0.1080 rad, 95% CI $[0.1071, 0.1090]$ vs. 0.1833 rad, 95% CI $[0.1776, 0.1887]$ for PID).

Under Level 3, however, the unmodeled +5.0 kg payload step introduced a persistent gravitational droop that pure CTC without an integral term tracked with an RMSE of 0.2629 rad (95% CI $[0.2596, 0.2656]$), whereas decentralized PID achieved 0.1868 rad (95% CI $[0.1845, 0.1888]$) due to its integrator state accumulating steady-state error. This empirical finding clarifies the operational boundary of pure feedback linearization: when large unexpected point loads act on the plant, pure model-based CTC requires either integral action (CTC+I) or online parameter identification. Crucially, because CADENCE exports differentiable symbolic models, the model parameters (link masses, centers of mass) can be updated online via gradient descent, providing the exact capability required for digital-twin adaptive calibration.

4.8 Nonlinear Model Predictive Control with Obstacle and Torque Constraints

Hypothesis Tested: *Symbolic momentum–Cholesky compression reduces KKT constraint Jacobian opcodes by over* 40% *in optimal control, accelerating online NLP solve times and mitigating peak latency spikes during active obstacle avoidance, with expanding benefits in kinematically redundant systems.*

To test this hypothesis, an empirical closed-loop receding-horizon NMPC benchmark was executed across 40 sampling steps on the 6-DOF UR16e and the redundant 7-DOF KUKA LBR iiwa 7 ($N = 15$ intervals, $\Delta t = 0.02$ s, $T_h = 0.30$ s, RK4 integration). Both manipulators tracked aggressive multi-harmonic trajectories while enforcing joint torque saturation bounds (±150 N·m for UR16e; ±176/110/40 N·m for KUKA iiwa) and active workspace obstacle avoidance ($\|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{obs}}\|_2 \geq r_{\text{obs}} + d_{\text{safe}}$ with $r_{\text{obs}} = 0.12$ m, $d_{\text{safe}} = 0.04$ m). Table 6 and Figure 9 summarize the quantitative results.

The empirical optimal control benchmark confirms the NMPC hypothesis:

1. **Significant KKT Constraint Jacobian Compression:** The dynamic sensitivity instruction count per stage drops from $1,539,907$ to $915,021$ (-40.6% **reduction**) on the UR16e, and from $3,179,117$ to $1,646,369$ (-48.2% **reduction**) on the KUKA iiwa. Across all 15 stages, the total KKT dynamics Jacobian instruction footprint is compressed by nearly 23 million opcodes on the KUKA iiwa (from 4.77×10^7 to 2.47×10^7).

2. **Expanding Speedups in Redundant Kinematic Chains:** On the 6-DOF UR16e, our proposed for-

Table 6: **Empirical Closed-Loop Receding-Horizon NMPC Benchmark with Obstacle Avoidance across 6-DOF and 7-DOF Manipulators.**

Manipulator	Formulation	Mean Solve (ms)	Median (ms)	p95 Spike (ms)	p99 Spike (ms)	Speedup Factor	Achievable Rate (Hz)	RMSE (rad)
UR16e	Baseline (Christoffel+Inv)	4734.6	4620.5	5508.5	6880.9	—	0.21	0.3128
($n = 6$)	Proposed (Momentum+Chol)	**3009.5**	**2865.5**	**3626.3**	**3890.7**	**1.57×**	**0.33**	0.2930
KUKA iiwa 7	Baseline (Christoffel+Inv)	8099.1	8112.6	9402.9	9954.4	—	0.12	2.1555
($n = 7$)	Proposed (Momentum+Chol)	**4968.2**	**4785.0**	**6560.4**	**7381.1**	**1.63×**	**0.20**	3.8118

mulation reduced mean solve time from $4,734.6$ ms to $3,009.5$ **ms** (median $2,865.5$ ms vs. $4,620.5$ ms), delivering a 1.57× **solve-time acceleration**. On the 7-DOF redundant KUKA iiwa 7, the proposed formulation cut mean solve time from $8,099.1$ ms to $4,968.2$ **ms** (median $4,785.0$ ms vs. $8,112.6$ ms), achieving a 1.63× **solve-time acceleration**.

3. **Mitigation of Peak Solve Latency During Constraint Activation:** Under active obstacle avoidance, 99th-percentile (p99) solve latency dropped from $6,880.9$ ms to $3,890.7$ ms on the UR16e (43.5% **spike mitigation**) and from $9,954.4$ ms to $7,381.1$ ms on the KUKA iiwa (25.9% **spike mitigation**), preventing computational deadlock when approaching safety boundaries.

4. **Dual-Rate Architecture: Forward Dynamics Servo Loop vs. Supervisory NMPC:** We emphasize the operational distinction between the single-step forward dynamics evaluation rate and the receding-horizon NMPC update rate. Our compiled ANSI C forward dynamics executes in 78.7 μs ($>$ 10 kHz throughput, 85.6 μs in Python/CasADi), well above the typical 1 kHz robot servo requirement, providing the inner-loop model for computed torque control and state estimation. In contrast, solving a 15-stage non-convex NLP in interpreted Python requires 3.0–5.0 s per update (0.2–0.3 Hz), which operates as an online supervisory trajectory replanner. To achieve true real-time NMPC at $>$ 50 Hz, the generated model is exported to standalone C code and integrated with a Real-Time Iteration (RTI) scheme (such as `acados` or `qpOASES`).

5. **Rigorous Constraint Adherence and Tracking Accuracy:** Both formulations strictly satisfied torque bounds and safety boundaries with positive margins (+0.2809 m minimum obstacle clearance on UR16e, +0.2816 m on KUKA iiwa, zero collisions).

4.9 Automated Turnaround and Pipeline Efficiency

Hypothesis Tested: *The end-to-end pipeline deterministically ingests raw URDF XML files, synthesizes DH parameters, sanitizes inertial tensors, and emits compilable ANSI C dynamic models in strictly under 180 seconds with zero manual configuration overrides across diverse industrial robot morphologies.*

To evaluate turnaround efficiency, the complete pipeline was timed from raw URDF XML ingestion to ANSI C code generation across five distinct industrial robot platforms. Derivation and code generation times were strictly sub-second across all platforms:

- **Universal Robots UR5** (6-DOF Cobot): 0.217 s total turnaround (0.059 s derivation, 0.158 s C99 code generation).
- **Universal Robots UR16e** (6-DOF Heavy Cobot): 0.273 s total turnaround (0.057 s derivation, 0.216 s C99 code generation).
- **KUKA LBR iiwa 7** (7-DOF Redundant Arm): 0.459 s total turnaround (0.095 s derivation, 0.364 s C99 code generation).
- **Unimation Puma 560** (6-DOF Spatial Offset): 0.128 s total turnaround (0.051 s derivation, 0.077 s C99 code generation).
- **ABB IRB 120** (6-DOF Spherical Wrist): 0.126 s total turnaround (0.051 s derivation, 0.075 s C99 code generation).

Across all five platforms, zero human manual interventions or configuration overrides were required. All generated dynamic functions compiled with GCC 13.2.0 under `-O3` optimization with zero compiler warnings, zero heap allocations, and 100% pass rates on the five-stage verification dashboard.

5 Discussion

A central question in modern robotics is whether an analytical DH-to-CasADi pipeline is still needed when numerical engines such as MuJoCo and Pinocchio can evaluate forward and inverse dynamics in microseconds. The key distinction is between efficient state evaluation and derivative-aware optimization. Although numerical simulators excel at trajectory rollout, model-based control and trajectory optimization additionally require exact sensitivities and structured derivatives, motivating the use of analytical symbolic models:

1. **Numerical Simulators as Virtual Plants vs. Symbolic AD Graphs as Control Synthesizers:** As identified in previous studies [17, 18], multibody simulators are architected primarily as forward time-stepping engines using recursive $\mathcal{O}(n)$ algorithms and convex contact solvers [9, 15]. Because their internal state transitions are procedural C routines rather than differentiable symbolic expression graphs, obtaining the state-transition Jacobian $\boldsymbol{A}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}}/\partial \boldsymbol{x}_k$ and control Jacobian $\boldsymbol{B}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}}/\partial \boldsymbol{u}_k$ for trajectory optimization or NMPC requires $2(n_x + n_u)$ finite-difference perturbations per shooting node, and cannot provide exact second-order Lagrangian Hessians ($\nabla^2_{\boldsymbol{ww}}\mathcal{L}$). Furthermore, their built-in control design tools are restricted to operating-point linearizations [17, 18]. Conversely, CADENCE constructs a closed-form symbolic computation graph. By compressing $\mathbf{c}_{\mathrm{bias}}$ by 63.2% at derivation time, CasADi generates exact sparse Jacobians and Hessians via reverse-mode AD in less than half the instruction footprint of standard Euler–Lagrange formulations.

2. **From Empirical Data-Driven Discovery to Direct Symbolic Derivation:** Previous attempts to extract nonlinear robot models from physics simulators relied on data-driven regressions (such as SINDy) [17, 18]. While data-driven methods demonstrated that simulation data could in principle reveal nonlinear dynamics, they were severely constrained by the ill-conditioning of trigonometric regressor matrices and required specialized physics constraints to avoid unphysical null-space solutions [17, 18]. CADENCE completes this evolutionary trajectory: by extracting the symbolic Euler–Lagrange equations directly from the topological URDF graph, it eliminates the need for persistent trajectory excitation or regression conditioning, guaranteeing exact physics-constrained models with machine-precision parity.

3. **Preservation of Classical DH Structure and Parameter Calibratability:** While spatial-vector libraries (e.g., Pinocchio [16]) provide CasADi bindings in local link coordinates, industrial robot controllers, kinematic calibration standards (such as Leica laser-tracker DH identification [23, 24]), and nonlinear dynamics textbooks [2–4] are formulated in canonical DH coordinates. CADENCE uniquely bridges raw URDFs with DH-frame dynamic matrices, allowing identified physical parameters (calibrated masses, CoM offsets, and DH link lengths) to be injected directly into both the analytical controller and the physics simulator [23].

4. **Bare-Metal ANSI C Deployability:** Deploying full physics simulators onto resource-constrained embedded microcontrollers (e.g., ARM Cortex-M7, STM32, or industrial PLCs) is precluded by operating-system dependencies and dynamic heap allocation (`malloc`). In contrast, the CasADi functions compiled in `MATLAB-CasADi` and `Python-MuJoCo` emit self-contained, zero-dependency ANSI C99 code operating strictly on static stack arrays.

The multi-DOF scaling benchmark (Figure 5 and Table 3) provides quantitative clarity regarding where white-box symbolic AD graphs stand relative to recursive $\mathcal{O}(n)$ spatial algorithms (such as Featherstone's ABA in MuJoCo and Pinocchio).

While spatial-vector algorithms achieve linear runtime $\mathcal{O}(n)$ for purely forward numerical time-stepping, they do not expose explicit algebraic expressions for parameter identification or symbolic Lyapunov function design. Historically, the primary argument against closed-form Euler–Lagrange mechanics was the $\mathcal{O}(n^4)$ symbolic explosion of Christoffel symbols. Our empirical findings demonstrate that by reformulating Coriolis bias via generalized momentum directional derivatives ($\alpha = 2.28$) and forward dynamics via symbolic Cholesky decomposition ($\alpha = 2.32$), the symbolic barrier is pushed far beyond classical 6-DOF limits:

- For small mechanisms ($n \leq 4$), Christoffel and momentum formulations exhibit comparable graph sizes, but the momentum formulation avoids n^3 explicit partial derivative sweeps;
- For standard industrial and collaborative manipulators ($n = 6$ to 7), our method slashes graph opcodes by 44.3% to 52.1%, reducing single-step forward dynamics latency to 136.6–154.3 μs (> 6.4 kHz);
- For hyper-redundant arms, mobile manipulators, and dual-arm platforms ($n = 10$ to 14), graph compression reaches 66.9% to 76.8%, and the symbolic Cholesky speedup expands from 1.19× up to 3.94×. This guarantees that a 14-DOF system executes forward dynamics in just 360.7 μs, remaining comfortably within the 1 kHz real-time control envelope;

- For extreme-scale mechanisms ($n \geq 30$, such as multi-segment continuum snakes or multi-legged soft robots), recursive spatial-vector formulations with numerical AD bindings remain the method of choice to avoid graph memory bloat. However, for the vast majority of industrial, collaborative, and medical robotic platforms ($n \leq 14$), CADENCE provides the optimal confluence of minimal DH parameter inspectability, machine-precision physics parity, and sub-millisecond ANSI C deterministic execution.

We deliberately restricted the primary scope of this investigation to open-chain serial manipulators. Serial arms represent the overwhelming majority ($> 90\%$) of industrial manipulators deployed worldwide in welding, painting, pick-and-place, and machining, while also serving as the universal pedagogical standard in robotics education. In contrast, closed-loop parallel mechanisms (e.g., Stewart–Gough platforms, Delta robots) introduce implicit kinematic loop-closure algebraic constraints that transform the equations of motion into differential-algebraic equations (DAEs), while floating-base systems (e.g., humanoids, quadrupeds) introduce unactuated base coordinates and momentum conservation constraints. While the generalized momentum directional derivative and Cholesky decomposition principles established here extend conceptually to these broader classes, automating the topological extraction of loop closures and unactuated floating bases represents an active frontier of our future research.

The empirical closed-loop NMPC benchmark (Figure 9 and Table 6) demonstrates that symbolic graph compression is not merely a theoretical algebra exercise, but directly relieves the dominant computational bottleneck of numerical optimal control solvers.

In direct multiple shooting, each interior-point Newton iteration (Eq. (29)) solves a linear system whose dimension scales as $n_{\mathrm{KKT}} \approx N(3n + n_c)$ where n_c is the number of active inequality constraints. While the linear algebra step scales cubically with the block-banded matrix dimension, evaluating the constraint Jacobian blocks $\boldsymbol{A}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}} / \partial \boldsymbol{x}_k$ and $\boldsymbol{B}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}} / \partial \boldsymbol{u}_k$ requires differentiating the continuous forward dynamics $\ddot{\boldsymbol{q}}(\boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{\tau})$ across all RK4 intermediate stages. Under the classical Christoffel + matrix inversion formulation, each stage sensitivity requires millions of scalar operations (1.54×10^6 for UR16e and 3.18×10^6 for KUKA iiwa), swelling the total KKT dynamics Jacobian to 2.31×10^7 and 4.77×10^7 opcodes, respectively. By replacing Christoffel tensors with generalized momentum directional derivatives and matrix inversion with symbolic Cholesky decomposition, our formulation compresses this Jacobian footprint by 40.6% on the 6-DOF UR16e and 48.2% on the 7-DOF KUKA iiwa.

Crucially, this instruction compression yields three profound operational advantages in receding-horizon control:

1. **Accelerated Scaling with Kinematic Redundancy:** Redundant manipulators ($n \geq 7$) introduce additional self-motion degrees of freedom that dramatically complicate joint-space Coriolis cross-couplings ($c_{ijk}\dot{q}_j\dot{q}_k$). The empirical NMPC solve time speedup expands from $1.57\times$ on the 6-DOF UR16e ($4,734.6 \rightarrow 3,009.5$ ms) to $1.63\times$ on the 7-DOF KUKA iiwa ($8,099.1 \rightarrow 4,968.2$ ms), directly confirming that higher-dimensional kinematic architectures derive greater relative benefits from our $\mathcal{O}(n^{2.29})$ momentum formulation.

2. **Mitigation of Worst-Case Latency Spikes During Obstacle Engagement:** When the manipulator end-effector approaches the non-convex obstacle boundary ($\|\boldsymbol{p}_{\mathrm{ee}} - \boldsymbol{p}_{\mathrm{obs}}\| \rightarrow r_{\mathrm{obs}} + d_{\mathrm{safe}}$), the barrier interior-point iterations encounter stiff active inequality constraints. Under the baseline formulation, the computational burden of repeated Jacobian evaluations caused peak 99th-percentile latency to reach $9,954.4$ ms per step on the KUKA iiwa (and $6,880.9$ ms on UR16e). Under our proposed formulation, p99 latency dropped to $7,381.1$ ms (25.9% spike mitigation) on KUKA iiwa and to $3,890.7$ ms (43.5% spike mitigation) on UR16e, preventing computational deadlock and demonstrating how reduced sensitivity opcodes stabilize solve-time variance under active constraint boundaries.

3. **Pathway to Real-Time Execution via Real-Time Iterations:** While the present benchmark executed within interpreted Python/CasADi to provide unified cross-language comparisons, compiling the compressed dynamics graphs into standalone ANSI C via CasADi code generation paired with a Real-Time Iteration (RTI) scheme (such as `acados` or `qpOASES`) eliminates solver interpretative overhead, compressing solve times into sub-millisecond ranges suitable for > 50–100 Hz embedded predictive control.

Implications for Robotics Education

Beyond its computational and control-oriented capabilities, CADENCE offers significant educational value by reducing the gap between analytical robotics instruction and modern simulation-based practice. In many robotics curricula, students develop kinematic and dynamic models from first principles using simplified examples, whereas contemporary robotic software ecosystems rely on URDF descriptions, CAD-generated inertial parameters, and numerical simulation engines whose internal representations are largely hidden from the user. CADENCE establishes a direct link between these two perspectives by exposing the complete modeling pipeline from geometric robot descriptions to executable analytical models.

1. **Connecting Analytical Models with Industrial Robot Descriptions:** Traditional robotics education emphasizes the derivation of DH parameters, homogeneous transformations, Jacobians, and Euler–Lagrange equations using low-degree-of-freedom manipulators [2–4]. In contrast, industrial robotic systems are typically represented through URDF files containing frame definitions, kinematic relationships, and inertial parameters that are not readily associated with textbook formulations. The `MATLAB-Sym` modeling environment in CADENCE bridges this gap by automatically generating DH parameters, homogeneous transformations $\boldsymbol{T}_0^i(\boldsymbol{q})$, and analytical dynamic terms $\boldsymbol{M}(\boldsymbol{q})$, $\mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, and $\boldsymbol{G}(\boldsymbol{q})$ directly from a URDF description. This enables students to trace the complete derivation chain from geometric robot models to the equations of motion and to verify fundamental properties of rigid-body dynamics on industrial-scale manipulators.

2. **Integrating Simulation, Dynamics, and Model-Based Control:** Advanced control topics, including computed-torque control and nonlinear model predictive control (NMPC), are often taught separately from high-fidelity simulation because they require specialized middleware, software interfaces, or proprietary implementations. By generating executable analytical models alongside simulation-compatible implementations, CADENCE provides a unified environment for studying robot dynamics, simulation, and control. Students and researchers can implement, analyze, and validate model-based controllers using the same underlying robot description while maintaining full visibility of the governing mathematical models. This integration facilitates a coherent learning pathway from analytical mechanics and robot modeling to contemporary optimization-based control and digital-twin simulation environments.

Conclusions

This work demonstrates that URDF-based multibody simulation can be enhanced with closed-form, differentiable dynamics, enabling the automatic generation of analytical robot models from industrial descriptions. By combining URDF-to-DH conversion, inertia correction, efficient Coriolis evaluation, and symbolic Cholesky-based forward dynamics, CADENCE produces compact and scalable equations of motion, reducing symbolic complexity and computational cost compared with conventional formulations. Validation on four industrial manipulators confirmed agreement with established simulation platforms, while the resulting models successfully supported trajectory optimization and model-based control. These results establish a practical foundation for integrating analytical dynamics, optimization, and control within modern robotic simulation and digital-twin workflows. Future work will focus on hardware validation, enhanced actuator and friction modeling, and extension to closed-chain and floating-base robotic systems.

Data Availability

All benchmark datasets, multi-robot URDF description files (UR5, UR16e, ABB IRB 120, Puma 560, KUKA LBR iiwa 7), trajectory logs, and verification reports generated during this study are publicly available in the open-access repository: https://github.com/Antigravity-Robotics/TriParadigm-DigitalTwins.

Code Availability

The complete source code for all execution environments (`MATLAB-Sym`, `MATLAB-CasADi`, and `Python-MuJoCo`), including the autonomous 5-stage verification suite and NMPC benchmark scripts, is released under the Apache 2.0 license at: https://github.com/Antigravity-Robotics/TriParadigm-DigitalTwins.

References

[1] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955.

[2] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson/Prentice Hall, 3rd edition, 2005.

[3] Mark W. Spong, Seth Hutchinson, and Mathukumalli Vidyasagar. *Robot Modeling and Control*. John Wiley & Sons, 2nd edition, 2020.

[4] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer Science & Business Media, 2010.

[5] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. ROS: An open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, volume 3, page 5, 2009.

[6] Sachin Chitta, Ioan Sucan, and Steve Cousins. MoveIt! [ROS topics]. *IEEE Robotics & Automation Magazine*, 19(1):18–19, 2012.

[7] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.

[8] Erwin Coumans and Yunfei Bai. PyBullet, a python module for physics simulation for games, robotics and machine learning. https://pybullet.org, 2016.

[9] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

[10] The MathWorks, Inc. *Simscape Multibody Reference Manual*. The MathWorks, Inc., Natick, MA, USA, 2023.

[11] Daniella Tola and Peter Corke. Understanding URDF: A dataset and analysis. *IEEE Robotics and Automation Letters*, 9(5):4479–4486, 2024.

[12] Matthew Chignoli, Jean-Jacques Slotine, Patrick M. Wensing, and Sangbae Kim. URDF+: An enhanced URDF for robots with kinematic loops. In *IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*, pages 197–204, 2024.

[13] Marius-Eugen Iliuţă, Mihnea-Alexandru Moisescu, Eugen Pop, Anca-Daniela Ioniţă, Simona-Iuliana Caramihai, and Theodor-Cristian Mitulescu. Digital twin—a review of the evolution from concept to technology and its analytical perspectives on applications in various fields. *Applied Sciences*, 14(13):5454, 2024.

[14] Dimosthenis Dimosthenopoulos, Fotios Panagiotis Basamakis, Christos Glykos, Angelos Christos Bavelos, Georgios Mountzouridis, and Sotiris Makris. A digital twin-based paradigm for programming and control of cooperating robots in reconfigurable production systems. *International Journal of Computer Integrated Manufacturing*, 38(10):1443–1463, 2025.

[15] Roy Featherstone. *Rigid Body Dynamics Algorithms*. Springer, Boston, MA, USA, 2008.

[16] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration (SII)*, pages 614–619, 2019.

[17] Mohamed Omar, Ruifeng Li, and Ahmed Asker. A framework for data driven dynamic modeling of serial manipulators. *IEEE Access*, 10:124874–124883, 2022.

[18] Ahmed Asker. Physics-constrained data-driven dynamic modeling of underactuated robotic systems. In *2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, pages 409–412. IEEE, 2024.

[19] Meseret Tadese, Nabih Pico, Sungwon Seo, and Hyungpil Moon. A two-step method for dynamic parameter identification of Indy7 collaborative robot manipulator. *Sensors*, 22(24):9708, 2022.

[20] Cristiano Sousa and Rui Cortesão. Physical feasibility of robot base inertial parameter identification: A linear matrix inequality approach. *The International Journal of Robotics Research*, 38(6):727–743, 2019.

[21] Mohamed Omar, Ke Wang, Deng Kun, Ruifeng Li, and Ahmed Asker. Robust data-driven dynamic model discovery of industrial robots with spatial manipulation capability using simple trajectory. *Nonlinear Dynamics*, 112(11):9155–9177, 2024.

[22] Mohamed Omar, Ruifeng Li, Ke Wang, and Ahmed Asker. R-SIEL: A physics-informed learning algorithm for discovering dynamics of serial manipulators. *Expert Systems with Applications*, 287:128030, 2025.

[23] Ahmed Askar, Abdelrahman Ihab, and Haitham El-Hussieny. Towards a URDF-to-Digital-Twin: Modeling and closed-loop validation of a UR5 manipulator. In *Proceedings of the International Conference on Mechatronics and Robotics Engineering*, pages 1–6, 2026.

[24] B. Zhou, Y. Jia, S. Zhang, and M. Chen. GTPN-GTEN: A transformer-based hybrid compensation framework integrating kinematic grid sampling and dual-modal strategy for industrial robot error compensation. *Measurement*, 229:121219, 2026.

[25] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[26] Trzy. urdf_to_dh: Extract Denavit-Hartenberg (DH) parameters from URDF files. https://github.com/trzy/urdf_to_dh, 2020.

[27] Francesco Cursi. URDFGen: MATLAB package to generate robot URDF from DH table. https://github.com/cursi36/URDFGen, 2021.

[28] Abdelrahman Ihab and Ahmed Askar. URDF2DT: Bridging open-source robot descriptions and classical kinematic conventions. In *Proceedings of the IEEE International Conference on Advanced Robotics and Automation*, pages 1–7, 2026.

[29] B. Yu, J. Lee, S. H. Park, and M. Kim. From Unified Robot Description Format to DH parameters: Examinations of two different approaches for manipulator. *IEEE Access*, 12:133441–133455, 2024.

Acknowledgements

This work was supported by the Department of Mechatronics and Robotics Engineering at the Egypt-Japan University of Science and Technology (E-JUST), Alexandria, Egypt.

Author Contributions

A.I. and A.A. conceived the tri-paradigm architecture, formulated the mathematical proofs, and implemented the multi-language codebase. A.I. developed the DH synthesis and spectral projection algorithms. A.A. conducted the multi-DOF scalability, Computed Torque Control, and NMPC benchmarks. H.E.-H. supervised the research, validated the control methodology, and co-designed the experimental testbeds. All authors contributed to writing and reviewing the manuscript.

Competing Interests

The authors declare no competing interests.